

Resolución de Problemas Algorítmicos

Un protocolo metódico y universal para resolver problemas de algoritmos, estructuras de datos y POO — desde los básicos hasta retos tipo HackerRank y LeetCode.

ENFOQUE

**Framework de 6 pasos
+ reconocimiento de
patrones**

LENGUAJE

**Dart (templates listos
para usar)**

NIVEL

Intermedio a Avanzado

TIEMPO ESTIMADO

**25–35 horas (con
práctica)**

Big-O

Patrones

Recursión

System Design

Contenido

12 capítulos · Protocolo de Resolución de Problemas Algorítmicos en Dart

PARTE 1 · METODOLOGÍA

01	01 — Metodología General: Framework de 6 Pasos
02	02 — Cómo analizar la complejidad de un problema
03	03 — Reconocimiento de Patrones

PARTE 2 · ESTRUCTURAS Y PATRONES

04	04 — Estructuras de Datos: Referencia Rápida
05	05 — Patrones Avanzados con Templates en Dart

PARTE 3 · COMUNICACIÓN Y PRÁCTICA

06	06 — Comunicación y Pseudocódigo
07	07 — Ejercicios de Práctica

PARTE 4 · RECURSIÓN Y SYSTEM DESIGN

09	09: Recursión y Backtracking
10	10: System Design Básico para Flutter

PARTE 5 · ERRORES Y MÁS PRÁCTICA

11	11: Errores Comunes al Resolver Problemas
12	12: Ejercicios Adicionales (+25 problemas)

PARTE 6 · CIERRE

08	08 — Recursos Externos
----	------------------------

01 — Metodología General: Framework de 6 Pasos

Antes de escribir una sola línea de código, sigue estos 6 pasos en orden. Saltar cualquiera de ellos es la causa #1 de soluciones incorrectas o lentas.

El Framework

PASO 1: Entender el Problema
"¿Qué me piden? ¿Qué me dan?"

PASO 2: Analizar Constraints
"¿Qué complejidad necesito?"

PASO 3: Explorar Ejemplos y Edge Cases
"¿Qué pasa con casos extremos?"

PASO 4: Identificar el Patrón
"¿Qué algoritmo/estructura encaja aquí?"

PASO 5: Escribir Pseudocódigo
"¿Puedo explicar la lógica en palabras?"

PASO 6: Implementar, Testear y Optimizar
"¿Funciona? ¿Es eficiente?"

PASO 1: Entender el Problema

Lee el enunciado **dos veces**. No asumas nada. Responde estas preguntas:

1. **¿Cuál es la entrada?** (¿array? ¿string? ¿grafo? ¿grid? ¿número?)
2. **¿Cuál es la salida esperada?** (¿un número? ¿un booleano? ¿una lista? ¿un índice?)
3. **¿Qué significa exactamente cada término?** (Si dice "movimiento", define qué es un movimiento)
4. **¿Hay restricciones implícitas?** (¿circular? ¿orden importa? ¿puede haber negativos?)

Truco: Reformula el problema con tus propias palabras. Si no puedes explicarlo simple, no lo entiendes.

PASO 2: Analizar Constraints

Las constraints son la **señal más fuerte** para elegir algoritmo. Lées **antes** de pensar en la solución.

La regla clave: un ordenador ejecuta $\sim 10^8$ operaciones por segundo. Si tu algoritmo hará más de eso, será demasiado lento.

Tabla completa de constraints → complejidad: Ver [02-analisis-complejidad.md](#).

En resumen:

- $n \leq 10$ → backtracking completo
- $n \leq 5.000$ → dos loops anidados ($O(n^2)$)
- $n \leq 10^5$ → sorting o binary search ($O(n \log n)$)
- $n \leq 10^6$ → un solo recorrido ($O(n)$)

PASO 3: Explorar Ejemplos y Edge Cases

Antes de diseñar el algoritmo, trabaja **a mano** al menos 3 ejemplos:

- Caso normal** — el ejemplo del enunciado
- Edge case** — input vacío, un solo elemento, todos iguales
- Caso tricky** — negativos, duplicados, valores extremos

Edge Cases más comunes

Tipo de input	Edge cases a verificar
Array/String	Vacío, un elemento, todos iguales, todos negativos
Número	0, 1, negativo, overflow
Árbol	Vacío, un nodo, degenerado (lineal), completo
Grafo	Sin aristas, todos conectados, ciclos
Grid	1×1, todo bloqueado, todo abierto

PASO 4: Identificar el Patrón

Este es el paso **más importante**. No intentes inventar un algoritmo desde cero. Usa el reconocimiento de patrones (detallado en [03-reconocimiento-patrones.md](#)).

Preguntas guía

- ¿Cuál es el tipo de input? → Array, String, Graph, Tree, Grid, etc.

2. **¿Qué me piden?** → ¿Min/max? ¿Todos los resultados? ¿Sí/no? ¿Un camino?
3. **¿Qué keywords aparecen?** → "contiguo", "mínimo pasos", "par", "ciclo", "k-ésimo"

Si puedes nombrar el patrón antes de codificar, ya tienes la mitad de la solución.

PASO 5: Escribir Pseudocódigo

Nunca saltes directamente al código. Escribe pseudocódigo primero. Esto te permite:

- Validar la lógica **sin** lidiar con sintaxis
- Detectar errores lógicos antes de implementar
- Comunicar tu idea a otros (entrevista, code review)

Ejemplo de pseudocódigo

PROBLEMA: Validar que los paréntesis están balanceados

PATRÓN: Stack (LIFO) – el último abierto debe ser el primero en cerrarse

1. Crear un stack vacío
2. Para cada carácter en el string:
 - a. Si es '(' o '[', agregar al stack
 - b. Si es ')' o ']', verificar que el stack no esté vacío
 - Si está vacío → retorno false (cerrar sin abrir)
 - Si el tope del stack no coincide → retorno false
 - Si coincide → sacar del stack
3. Si el stack está vacío → retorno true (todo balanceado)
Si no → retorno false (quedaron abiertos sin cerrar)

PASO 6: Implementar, Testear y Optimizar

Implementar

- Usa nombres de variables descriptivos
- Maneja edge cases primero (early returns)
- Usa la estructura de datos correcta

Testear (mínimo 3 casos)

1. **Happy path** — el caso normal del enunciado
2. **Edge case** — input vacío, un elemento
3. **Stress case** — input grande mental para validar complejidad

Optimizar

Pregúntate: "¿Puedo hacer mejor?" Solo si el brute force es correcto pero lento, busca la optimización.

Resumen del Framework

Entender → Constraints → Ejemplos → Patrón → Pseudocódigo → Código

NO hagas:

- × Codificar sin entender el problema
- × Ignorar las constraints
- × Probar solo el caso happy path
- × Adivinar el patrón sin analizar señales
- × Saltar del enunciado al código directamente

Ejemplo rápido: Valid Parentheses

Problema: Dado un string con solo `(, ), {, }, [, y ]`, determinar si los paréntesis están balanceados.

Paso 1: Entrada: string de paréntesis. Salida: booleano.

Paso 2: $n \leq 10^4 \rightarrow$ necesitamos $O(n)$. Un solo recorrido es suficiente.

Paso 3: Edge cases: string vacío $\rightarrow$ true, solo un carácter $\rightarrow$ false, `((` $\rightarrow$ false.

Paso 4: "Paréntesis balanceados" + "el último abierto debe ser el primero en cerrar" $\rightarrow$ **Stack** (LIFO).

Paso 5:

1. Stack = []
2. Para cada char en el string:
 - a. Si es apertura '(', '{', '[' $\rightarrow$ agregar al stack
 - b. Si es cierre ')', '}', ']':
 - Si stack vacío $\rightarrow$ return false
 - Si tope del stack no coincide $\rightarrow$ return false
 - Si coincide $\rightarrow$ sacar del stack
3. Return stack.isEmpty

Paso 6: Implementar, testear con `"("  $\rightarrow$  false ✓, "()"  $\rightarrow$  true ✓`

02 — Cómo analizar la complejidad de un problema

Cuando ves un problema en HackerRank, LeetCode o similar, necesitas saber si tu solución es **rápida antes de escribirla**. Esta guía te enseña el proceso completo, paso a paso, con ejemplos reales.

PARTE 0: Big-O desde cero (para principiantes absolutos)

Si nunca has escuchado de "Big-O" o "complejidad algorítmica", empieza aquí. Esta sección explica todo desde el principio.

¿Qué es Big-O?

Big-O es una forma de medir **qué tan rápido crece** el tiempo de ejecución de un algoritmo cuando aumentan los datos de entrada.

No es una fórmula matemática complicated — es una **clasificación** que nos dice:

- Si un algoritmo es **rápido** (funciona con muchos datos)
- Si un algoritmo es **lento** (solo funciona con pocos datos)

La analogía del supermercado

Imagina que estás en la fila del supermercado:

Situación	Tiempo que toma	Tipo de complejidad
Caja rápida (1 persona atendiendo)	1 minuto por cliente	$O(n)$ - lineal
Caja lenta (la persona busca todo lentamente)	10 minutos por cliente	$O(n)$ pero con constante grande
Caja automática (máquina nueva)	30 segundos por cliente	$O(1)$ - constante
Problema: si hay 100 personas...	La caja lenta tarda 1000 minutos	¡16 horas!

Big-O nos ayuda a predecir qué pasará cuando haya MUCHOS datos.

¿Por qué importa?

Ejemplo real:

- Tu solución funciona con 10 números ✓
- El problema tiene 100,000 números
- Si tu solución es **lenta**, recibes "Time Limit Exceeded"
- **Pierdes tiempo** porque no analizaste antes

Las 4 complejidades principales (explicadas simple)

1. $O(1)$ — Constante: "Siempre lo mismo"

Ejemplo: Buscar tu nombre en la guía telefónica si sabes que empieza con "M" → vas directo a esa sección.

En código: Acceder al primer elemento de un array: `arr[0]`

Cuándo la ves: Operaciones que **no dependen del tamaño** de los datos.

2. $O(n)$ — Lineal: "Uno por uno"

Ejemplo: Revisar una lista de tareas completadas → lees cada tarea una por una.

En código:

```
for (int i = 0; i < n; i++) {  
    // algo con cada elemento  
}
```

Cuándo la ves: Cuando recorres **todos los elementos una vez**.

3. $O(n^2)$ — Cuadrático: "Comparar cada cosa con cada cosa"

Ejemplo: Comparar cada libro de una estantería con todos los demás para ver si hay duplicados.

En código:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        // comparar i con j  
    }  
}
```

Cuándo la ves: Cuando tienes **dos loops anidados** que recorren los mismos datos.

4. $O(\log n)$ — Logarítmica: "Dividir por la mitad"

Ejemplo: Buscar una palabra en el diccionario: abres por la mitad, decides izquierda o derecha, repites.

En código:

```
while (n > 1) {  
  n = n ~/ 2;  // Divides por 2 en cada paso  
}
```

Cuándo la ves: Cuando en cada paso **descartas la mitad** de los datos.

Tabla resumen para memorizar

Complejidad	Nombre	Ejemplo cotidiano	¿Cuándo la ves en código?
O(1)	Constante	Abrir un libro en una página marcada	<code>arr[0]</code> , <code>map[key]</code>
O(n)	Lineal	Leer un libro página por página	Un solo <code>for</code> que recorre todo
O(n²)	Cuadrático	Comparar cada página con todas las demás	Dos <code>for</code> anidados
O(log n)	Logarítmica	Buscar en diccionario (mitad por mitad)	<code>while (n > 1) { n = n ~/ 2; }</code>

La pregunta clave de Big-O

Cuando ves un código, pregúntate:

"Si duplico la cantidad de datos (n), ¿qué pasa con el tiempo?"

Si duplicas n y el tiempo...	Entonces es...
Se duplica	O(n) — lineal
Se cuadruplica (4x)	O(n²) — cuadrático
Apenas cambia (+1 paso)	O(log n) — logarítmico
No cambia	O(1) — constante

¿Qué es una operación?

Antes de analizar complejidad, necesitas saber **qué cuenta como 1 operación**.

Regla simple: Cada vez que el procesador ejecuta una instrucción básica, es 1 operación.

Operaciones que son O(1) — siempre lo mismo

Operación	Ejemplo
Sumar, restar, multiplicar, dividir	<code>a + b</code> , <code>n * 2</code> , <code>a ~/ b</code>
Comparar dos valores	<code>a == b</code> , <code>a > b</code> , <code>a <= b</code>
Acceder a un array por índice	<code>arr[i]</code>
Acceder a un Map/Set por key	<code>map[key]</code> , <code>set.contains(x)</code>
Asignar un valor a una variable	<code>x = 5</code> , <code>contador++</code>
Imprimir	<code>print(x)</code>
Calcular módulo	<code>a % b</code>
Convertir número a string (número pequeño)	<code>n.toString()</code>

Operaciones que NO son O(1)

Operación	Complejidad	Ejemplo
Buscar en un array sin orden	O(n)	<code>arr.contains(x)</code> — recorre todo
Insertar al inicio de un array	O(n)	<code>arr.insert(0, x)</code> — mueve todo
Eliminar del medio de un array	O(n)	<code>arr.removeAt(i)</code> — mueve todo
Crear un array nuevo copiando otro	O(n)	<code>List.from(arr)</code>

¿Por qué importa esto?

Cuando analizas un loop, necesitas saber si las operaciones **dentro** del loop son O(1) o no.

Ejemplo 1: Operaciones O(1) dentro del loop → solo cuentas el loop

```
for (int i = 0; i < n; i++) {  
    total += arr[i];    // O(1) — suma  
    if (arr[i] > max)    // O(1) — comparación  
        max = arr[i];    // O(1) — asignación  
}  
// Total: n × 3 = O(n) ← las constantes se ignoran
```

Ejemplo 2: Operación O(n) dentro del loop → multiplicas

```
for (int i = 0; i < n; i++) {  
    if (arr.contains(arr[i] * 2))    // O(n) — búsqueda en array  
        print("Found");  
}  
// Total: n × n = O(n²) ← la búsqueda O(n) dentro multiplica
```

Truco rápido: Si ves `arr.contains()`, `arr.indexOf()`, o `arr.remove()` sin índice específico, probablemente es **$O(n)$** . Si ves `map[key]` o `set.contains()`, es **$O(1)$** .

¿Por qué me importa esto?

Imagina que resuelves un problema y tu solución funciona perfecto... con 10 datos. Pero cuando el judge de HackerRank prueba con 100,000 datos, recibes **Time Limit Exceeded**. Perdiste tiempo porque no analizaste la complejidad antes de codificar.

Big-O es la herramienta que te dice **cómo crece** el tiempo de tu algoritmo cuando crecen los datos. No es matemática abstracta — es la diferencia entre que tu solución pase o no pase.

El Proceso: 5 pasos

Cada vez que te enfrentes a un problema, sigue estos 5 pasos **en orden**:

PASO 1: Entender el problema
"¿Qué me piden? ¿Qué me dan?"

PASO 2: Descifrar las constraints
"¿Cuántos datos máximo tengo? → ¿Qué complejidad necesito?"

PASO 3: Analizar mi primera idea (brute force)
"¿Qué tan lenta es? → ¿Cumple con la complejidad?"

PASO 4: Optimizar (si es necesario)
"¿Puedo hacerlo más rápido? → ¿Con qué estructura?"

PASO 5: Verificar
"¿Funciona con edge cases? → ¿La complejidad es correcta?"

La pregunta clave: Si duplicas la cantidad de datos (n), ¿qué pasa con el tiempo?

Si duplicas n y el tiempo...	Entonces es...
Se duplica	$O(n)$ — lineal
Se cuadruplica	$O(n^2)$ — cuadrático
Apenas cambia	$O(\log n)$ — logarítmico
No cambia	$O(1)$ — constante

Ahora veamos el proceso completo con un problema real.

PARTE A: Ejemplo completo — Two Sum

Vamos a resolver el problema **Two Sum** de LeetCode/HackerRank juntos, siguiendo los 5 pasos. Cada concepto se explica en el camino.

El problema

Dado un array de enteros `nums` y un entero `target`, devuelve los **índices** de los dos números que suman `target`. Puedes asumir que hay exactamente una solución y no puedes usar el mismo elemento dos veces.

Ejemplo: `nums = [2, 7, 11, 15]`, `target = 9` → respuesta: `[0, 1]` porque `nums[0] + nums[1] = 2 + 7 = 9`.

PASO 1: Entender el problema

Leo el enunciado y respondo:

1. **¿Qué me dan?** Un array de números enteros y un número objetivo (target).
2. **¿Qué me piden?** Los **índices** de dos números que sumen el target.
3. **¿Qué restricciones hay?** No puedo usar el mismo elemento dos veces. Hay exactamente una solución.
4. **¿Puedo reformularlo?** "Encuentra dos números en el array que sumen el target y dime dónde están."

Truco: Si no puedes explicar el problema con tus propias palabras simples, no lo entiendes aún. Vuelve a leerlo.

PASO 2: Descifrar las constraints

Las constraints son la **señal más fuerte** para elegir tu enfoque. Siempre léelas antes de pensar en la solución.

En este problema, las constraints típicas son:

```
2 ≤ nums.length ≤ 104
-109 ≤ nums[i] ≤ 109
-109 ≤ target ≤ 109
```

¿Qué significa esto? La parte importante es `nums.length ≤ 104`. Eso es $n = 10,000$.

La regla de los 10⁸

¿Qué es 10⁸? Es notación científica para **100,000,000** (cien millones).

¿Por qué importa? Porque un procesador moderno ejecuta aproximadamente **100 millones de operaciones por segundo**. Si tu algoritmo hace más de eso, será demasiado lento.

La regla práctica: Si tu algoritmo hace **menos de 100 millones de operaciones**, funcionará en menos de 1 segundo.

Ejemplo concreto:

- Si $n = 10,000$ (diez mil):
 - $O(n) = 10,000$ operaciones → **0.0001 segundos** ✓ rápido
 - $O(n^2) = 100,000,000$ operaciones → **1 segundo** ⚠ justo en el límite
 - $O(n^3) = 10^{12}$ operaciones → **10,000 segundos** × demasiado lento

Tabla de referencia (memorízala):

Si el problema dice...	Tu algoritmo debe ser..	Operaciones máximas	Tiempo estimado
$n \leq 10$	Cualquier cosa	$10! = 3.6$ millones	0.03 segundos
$n \leq 100$	$O(n^3)$ o mejor	1,000,000	0.01 segundos
$n \leq 5,000$	$O(n^2)$ o mejor	25,000,000	0.25 segundos
$n \leq 100,000$	$O(n \log n)$ o mejor	1,700,000	0.02 segundos
$n \leq 10,000,000$	$O(n)$	10,000,000	0.1 segundos

¿Cómo usar esta tabla?

- Mira las constraints del problema (¿cuál es el valor máximo de n ?)
- Busca en la tabla qué complejidad necesitas
- Diseña tu algoritmo para cumplir esa complejidad

Ejemplo:

- Problema dice: $n \leq 100,000$
- Tabla dice: necesitas $O(n \log n)$ o mejor
- Tu algoritmo debe ser $O(n \log n)$, $O(n)$, $O(\log n)$, o $O(1)$

PASO 3: Analizar mi primera idea (brute force)

La primera idea que se nos ocurre es la más obvia: **probar todos los pares posibles**.

```
// Brute force: probar cada par de números
List<int> twoSum(List<int> nums, int target) {
    for (int i = 0; i < nums.length; i++) {           // ← Loop 1: recorre cada elemento
        for (int j = i + 1; j < nums.length; j++) {   // ← Loop 2: recorre todos los que quedan
            if (nums[i] + nums[j] == target) {
                return [i, j];
            }
        }
    }
    return [];
}
```

Ahora analicemos la complejidad (esto es lo importante)

Sigue este **checklist** cada vez que analices código:

Checklist: Cómo contar operaciones paso a paso

Paso 1: Buscar loops

- ☐ ¿Hay `for`? ¿Cuántos?
- ☐ ¿Hay `while`? ¿Cuántos?
- ☐ ¿Están **anidados** (uno dentro del otro) o **secuenciales** (uno tras otro)?

Paso 2: Contar iteraciones de cada loop

- ☐ Loop 1: ¿Cuántas veces se ejecuta?
- ☐ Loop 2: ¿Cuántas veces se ejecuta **POR CADA** iteración del Loop 1?

Paso 3: Contar operaciones DENTRO del loop

- ☐ ¿Hay sumas, restas, comparaciones? → $O(1)$ cada una
- ☐ ¿Hay búsquedas en Array con `.contains()` o `.indexOf()`? → $O(n)$ cada una
- ☐ ¿Hay búsquedas en Map/Set con `map[key]` o `set.contains()`? → $O(1)$ cada una

Paso 4: Calcular total

- ☐ Si los loops están **anidados**: `Loop1 × Loop2 × Operaciones dentro`
- ☐ Si los loops están **secuenciales**: `Loop1 + Loop2 + ...`

Paso 5: Simplificar

- ☐ Ignorar constantes (`3n` → `n`, `100n2` → `n2`)
- ☐ Quedarte con el término de mayor crecimiento

Aplicando el checklist a Two Sum

```

for (int i = 0; i < nums.length; i++) {           // ← Loop 1
  for (int j = i + 1; j < nums.length; j++) {     // ← Loop 2 (dentro del 1)
    if (nums[i] + nums[j] == target) {           // ← Operación
      return [i, j];
    }
  }
}

```

Paso	Pregunta	Respuesta
1	¿Cuántos loops?	2 <code>for</code>
2	¿Están anidados?	Sí — Loop 2 está dentro de Loop 1
3	Loop 1: ¿cuántas veces?	<code>n</code> veces (recorre todo el array)
4	Loop 2: ¿cuántas veces por cada del Loop 1?	Hasta <code>n</code> veces
5	Operaciones dentro del loop	<code>nums[i] + nums[j]</code> → $O(1)$, <code>==</code> → $O(1)$
6	Total	<code>n × n × 1 = n²</code>

Conclusión: $O(n^2)$ — complejidad cuadrática

¿Por qué ignorar constantes en Big-O?

Pregunta común: "Si mi código hace $3n$ operaciones y otro hace $100n$, ¿son iguales?"

Respuesta corta: En Big-O, sí. Ambos son $O(n)$.

¿Por qué? Porque Big-O mide el **patrón de crecimiento**, no la velocidad absoluta.

Ejemplo numérico:

n	3n	100n	Relación
10	30	1,000	33× más lento
1,000	3,000	100,000	33× más lento
1,000,000	3,000,000	100,000,000	33× más lento

Observación: La diferencia siempre es 33×, sin importar cuánto crezca n . Ambos crecen **linealmente**. Por eso decimos que ambos son $O(n)$.

La analogía: Es como clasificar carros por tipo (sedan, camioneta), no por velocidad. Un sedan a 100 km/h y otro a 200 km/h ambos son "sedanes". Big-O clasifica el **tipo de crecimiento**, no la velocidad exacta.

¿Cuándo SÍ importan las constantes? Cuando dos algoritmos tienen la **misma complejidad** (ambos $O(n)$), ahí sí comparas constantes. Pero primero optimiza la complejidad (de $O(n^2)$ a $O(n)$), luego optimiza constantes.

¿Por qué $O(n^2)$? (explicación visual)

Imagina un array de 4 elementos: `[2, 7, 11, 15]`

El Loop 1 recorre cada elemento:

- $i=0$: compara 2 con [7, 11, 15]
- $i=1$: compara 7 con [11, 15]
- $i=2$: compara 11 con [15]
- $i=3$: no compara con nada

Total de comparaciones: $3 + 2 + 1 = 6 = (4 \times 3) / 2$

Patrón: Para n elementos, son aproximadamente $n^2/2$ comparaciones. En Big-O ignoramos la constante $1/2$, así que es $O(n^2)$.

La regla práctica: Cuando ves `for` + `for` anidados que recorren los mismos datos, es $O(n^2)$.

¿Cumple con lo que necesito?

- Necesito: $O(n)$ o mejor
- Mi solución: $O(n^2)$
- Con $n = 10,000$: serían 100,000,000 operaciones = **1 segundo exacto**

Está justo en el límite. Podría funcionar, pero es arriesgado. Si el problema tuviera $n \leq 100,000$, $O(n^2)$ sería 10 mil millones = 100 segundos. No pasaría.

¿Qué hago? Optimizo.

PASO 4: Optimizar

Mi solución actual: $O(n^2)$ — demasiado lento para $n = 10,000$ **Necesito:** $O(n)$ o mejor

Pregunta clave: ¿Cómo puedo encontrar el complemento más rápido?

El problema del brute force

En el brute force, para cada elemento `nums[i]`, recorro **todo** el array buscando el complemento `target - nums[i]`. Eso es $O(n)$ para cada elemento, y como hay n elementos $\rightarrow O(n^2)$.

La idea de la optimización

En vez de buscar el complemento, puedo recordar los valores que ya vi.

Si estoy en el elemento `7` y necesito un `2` (porque $7 + 2 = 9$):

- **Brute force:** Recorro todo el array buscando un 2 $\rightarrow O(n)$
- **Optimización:** Pregunto: "¿ya vi un 2 antes?" $\rightarrow O(1)$

¿Qué es un HashMap (Map en Dart)?

Un **HashMap** es una estructura de datos que guarda pares **llave-valor** y permite buscar por llave en **$O(1)$** (tiempo constante).

Analogía: Es como un diccionario:

- Buscas una palabra (llave) → encuentras su definición (valor) **instantáneamente**
- No necesitas leer el diccionario página por página

En código Dart:

```
Map<int, int> seen = {};  
seen[7] = 0; // Guardo: valor 7 está en índice 0  
seen.containsKey(7); // → true (búsqueda O(1))  
seen[7]; // → 0 (acceso O(1))
```

La solución optimizada

```
List<int> twoSum(List<int> nums, int target) {  
  Map<int, int> seen = {}; // ← HashMap: valor → índice  
  
  for (int i = 0; i < nums.length; i++) { // ← Solo un loop  
    int complemento = target - nums[i]; // Calculo qué necesito  
  
    if (seen.containsKey(complemento)) { // ¿Ya vi ese número? O(1)  
      return [seen[complemento]!, i];  
    }  
  
    seen[nums[i]] = i; // Guardo el valor actual para después  
  }  
  return [];  
}
```

Analizamos la nueva complejidad

¿Cuántas operaciones hace ahora?

1. **Un solo loop** que recorre `n` elementos → $O(n)$
2. **Dentro del loop:**
 - Resta: `target - nums[i]` → $O(1)$
 - Búsqueda en Map: `seen.containsKey(complemento)` → $O(1)$
 - Inserción en Map: `seen[nums[i]] = i` → $O(1)$

Total: `n × 3 = 3n` operaciones. Las constantes (3) se ignoran en Big-O.

Esto es $O(n)$ — complejidad lineal.

¿Por qué es $O(n)$?

- **Un solo loop** → $O(n)$
- **Las operaciones dentro del loop son $O(1)$** → no cambian la complejidad
- **La estructura (HashMap) es clave** → sin ella, la búsqueda sería $O(n)$ y el total sería $O(n^2)$

Comparación visual

Enfoque	Código	Complejidad	Con n = 10,000
Brute force	2 loops anidados	$O(n^2)$	100,000,000 ops (1 segundo)
HashMap	1 loop + Map	$O(n)$	10,000 ops (0.0001 segundos)

10,000 veces más rápido. Esa es la diferencia entre que tu solución pase o no pase.

PASO 5: Verificar

Antes de enviar, pruebo con casos extremos:

Caso normal: `nums = [2, 7, 11, 15]`, `target = 9` → `[0, 1]` ✓

Edge case — números negativos: `nums = [-3, 4, 3, 90]`, `target = 0` → `[0, 2]` ✓

Edge case — números repetidos: `nums = [3, 3]`, `target = 6` → `[0, 1]` ✓ (funciona porque guardamos el índice antes de verificar)

Edge case — array de 2 elementos: `nums = [1, 2]`, `target = 3` → `[0, 1]` ✓

Lección de Two Sum

Concepto	Lo que aprendimos
Constraints	$n \leq 10^4 \rightarrow$ necesito $O(n)$ o mejor
Brute force	2 loops anidados → $O(n^2)$ → no cumple
Optimización	HashMap para búsqueda $O(1)$ → $O(n)$ → cumple
Estructura clave	Map en Dart: <code>containsKey()</code> y <code>[]</code> son $O(1)$

Ahora que hemos visto el proceso completo una vez, vamos a practicarlo con 4 problemas más de HackerRank. Cada uno sigue los mismos 5 pasos.

PARTE B: Ejemplos guiados — 4 problemas de HackerRank

En esta sección repetimos el proceso de los 5 pasos con problemas reales. Cada problema te enseña algo nuevo sobre cómo analizar la complejidad.

Ejemplo 1: Beautiful Days at the Movies

Problema en HackerRank

El problema

Lily quiere ir al cine en días "beautiful" (hermosos). Un día es beautiful si $|\text{day} - \text{reverse}(\text{day})| \% k == 0$.

Dados los días i (inicio) y j (fin), y un divisor k , devuelve cuántos días beautiful hay en el rango.

Ejemplo: $i = 20$, $j = 23$, $k = 6$

- Día 20: $\text{reverse}(20) = 02 = 2 \rightarrow |20 - 2| = 18 \rightarrow 18 \% 6 = 0 \checkmark$ beautiful
- Día 21: $\text{reverse}(21) = 12 \rightarrow |21 - 12| = 9 \rightarrow 9 \% 6 = 3 \times$ no beautiful
- Día 22: $\text{reverse}(22) = 22 \rightarrow |22 - 22| = 0 \rightarrow 0 \% 6 = 0 \checkmark$ beautiful
- Día 23: $\text{reverse}(23) = 32 \rightarrow |23 - 32| = 9 \rightarrow 9 \% 6 = 3 \times$ no beautiful
- Respuesta: 2

PASO 1: Entender el problema

1. **¿Qué me dan?** Dos números (inicio y fin de un rango) y un divisor.
2. **¿Qué me piden?** Contar cuántos números en el rango cumplen la condición.
3. **¿Qué es reverse?** Un número al revés: $21 \rightarrow 12$, $202 \rightarrow 202$, $123 \rightarrow 321$.
4. **¿Qué es beautiful?** Cuando la diferencia absoluta entre el número y su reverso es divisible por k .

PASO 2: Descifrar las constraints

$$\begin{aligned} 1 &\leq i \leq j \leq 10^7 \\ 1 &\leq k \leq 10^9 \end{aligned}$$

¿Qué significa esto?

- i y j son los días de inicio y fin del rango
- $j \leq 10^7$ significa que el rango puede tener hasta **10,000,000 de días**
- k es el divisor (puede ser hasta 1,000,000,000)

La parte clave es $j \leq 10^7$. Eso define el tamaño máximo de nuestro problema.

¿Qué complejidad necesito?

Usando la tabla de la sección anterior:

- $n \leq 10,000,000 \rightarrow$ necesito **$O(n)$** o mejor

Tabla de verificación: | Complejidad | Operaciones con $n=10^7$ | ¿Cabe? | |---|---|---| | $O(n)$ | 10,000,000 | ✓
Sí (0.1 segundos) | | $O(n \log n)$ | 230,000,000 | ⚠ En el límite (2.3 segundos) | | $O(n^2)$ | 10^{14} | × No (10,000 segundos) |

Conclusión: Necesito **$O(n)$ o mejor**.

PASO 3: Analizar mi primera idea

La idea obvia: Recorrer cada día del rango, calcular si es beautiful, y contar.

```
int beautifulDays(int i, int j, int k) {
    int count = 0;
    for (int day = i; day <= j; day++) {        // ← Un solo loop
        int reversed = int.parse(day.toString().split('').reversed.join());
        if ((day - reversed).abs() % k == 0) {
            count++;
        }
    }
    return count;
}
```

Análisis paso a paso:

1. **¿Cuántos loops hay?** Solo **1 loop** (`for (int day = i; day <= j; day++)`)
2. **¿Cuántas veces se ejecuta?** Desde `i` hasta `j` inclusive
3. **¿Qué es "n" aquí?** $n = j - i + 1$ (la cantidad de días en el rango)
 - Ejemplo: si $i=20, j=23 \rightarrow n = 23 - 20 + 1 = 4$ días
4. **¿Qué hay dentro del loop?** Operaciones simples:
 - Convertir a string: $O(\text{longitud del número}) \approx O(1)$ para números $\leq 10^7$
 - Invertir string: $O(\text{longitud}) \approx O(1)$
 - Convertir a número: $O(\text{longitud}) \approx O(1)$
 - Calcular módulo: $O(1)$

Total: n iteraciones $\times O(1)$ por iteración = **$O(n)$**

¿Cumple con las constraints? Sí. Con $n = 10^7$, son 10 millones de operaciones. Cabe en menos de 1 segundo.

PASO 4: Optimizar

¿Necesito optimizar? No. $O(n)$ cumple con las constraints.

A veces **no necesitas optimizar**. Si tu solución ya cumple con la complejidad necesaria, está bien así. No todo problema requiere la solución más compleja.

PASO 5: Verificar

Caso normal: $i = 20, j = 23, k = 6 \rightarrow 2 \checkmark$

Edge case — un solo día: $i = 5, j = 5, k = 1 \rightarrow \text{reverse}(5) = 5 \rightarrow |5-5| = 0 \rightarrow 0 \% 1 = 0 \rightarrow 1 \checkmark$

Edge case — rango grande: $i = 1, j = 10^7, k = 1 \rightarrow$ todos son beautiful (cualquier cosa mod 1 = 0) $\rightarrow 10^7 \checkmark$

Lección de Beautiful Days

¿Qué aprendimos de este ejemplo?

1. **Identificar "n"**: En problemas con rangos, n = tamaño del rango ($j - i + 1$)
2. **Un solo loop = $O(n)$** : Cuando recorres un rango una sola vez, es lineal
3. **No siempre optimizar**: Si $O(n)$ cumple con las constraints, está perfecto
4. **Operaciones simples no cambian la complejidad**: Convertir a string, invertir, módulo $\rightarrow$ todo $O(1)$ o casi

Tabla resumen:

Concepto	Lo que aprendimos
Un solo loop	Un <code>for</code> que recorre un rango = $O(n)$
No siempre optimizar	Si $O(n)$ cumple, no busques algo más complejo
Operaciones dentro del loop	Si son simples (suma, módulo), no cambian la complejidad
Cómo calcular n	Para rangos: $n = \text{fin} - \text{inicio} + 1$

Patrón para problemas similares: Si ves un problema que dice:

- "Recorrer un rango de A a B"
- "Contar números que cumplan una condición"
- "Para cada elemento del array, hacer algo"

Probablemente sea **$O(n)$** con un solo loop.

Ejemplo 2: Jumping on the Clouds — Revisited

Problema en HackerRank

El problema

Un niño salta en nubes. Hay nubes normales (0) y nubes de trueno (1). El niño empieza en la nube 0 con energía `e`.

- Salta `k` posiciones $\rightarrow$ consume 1 de energía

- Si cae en nube de trueno (1) → consume 2 de energía extra
- El juego termina cuando vuelve a la nube 0

Ejemplo: $c = [0, 0, 1, 0, 0, 1, 1, 0]$, $k = 2$, $e = 100$

- Salta 0→2 (trueno): energía = $100 - 1 - 2 = 97$
- Salta 2→4: energía = $97 - 1 = 96$
- Salta 4→6 (trueno): energía = $96 - 1 - 2 = 93$
- Salta 6→0: energía = $93 - 1 = 92$
- Respuesta: 92

PASO 1: Entender el problema

1. **¿Qué me dan?** Un array de nubes (0 o 1), un tamaño de salto k , y energía inicial e .
2. **¿Qué me piden?** La energía restante al volver a la nube 0.
3. **¿Cómo salta?** Siempre de k en k posiciones, en un array circular.
4. **¿Qué cuesta?** 1 energía por salto + 2 extra si la nube es de trueno.

PASO 2: Descifrar las constraints

$$\begin{aligned} 2 &\leq n \leq 25 \\ 1 &\leq k \leq n \\ 0 &\leq e \leq 100 \end{aligned}$$

$n \leq 25$. Esto es **muy pequeño**. Casi cualquier complejidad funciona.

Complejidad	Operaciones	¿Cabe?
$O(n)$	25	✓ Sí
$O(n^2)$	625	✓ Sí
$O(2^n)$	33 millones	⚠ Mucho, pero n es tan pequeño que...

Con n tan pequeño, no necesitas preocuparte por la complejidad. Pero vamos a analizarla igual para practicar.

PASO 3: Analizar mi primera idea

La idea: **simular los saltos** hasta volver a la nube 0.

```
int jumpingOnClouds(List<int> c, int k, int e) {
    int energy = e;
    int pos = 0;

    do {
        pos = (pos + k) % c.length; // Saltar k posiciones (circular)
        energy--; // Cada salto cuesta 1

        if (c[pos] == 1) {
            energy -= 2; // Nube de trueno cuesta 2 extra
        }
    } while (pos != 0);

    return energy;
}
```

Análisis:

1. El loop ejecuta n / k iteraciones (salta de k en k posiciones hasta volver al inicio).
2. Dentro del loop: una suma, un módulo, una resta, una comparación. Todo $O(1)$.

¿Qué es "n" aquí? $n = c.length$ (el número de nubes).

Complejidad: $O(n/k)$ — pero como $k \geq 1$, en el peor caso es $O(n)$.

¿Cumple? Sí. Con $n \leq 25$, son máximo 25 iteraciones. Cabe sin problemas.

PASO 4: Optimizar

¿Necesito optimizar? No. $O(n)$ con $n \leq 25$ es instantáneo.

Pero vale la pena notar algo: **el módulo % crea un array circular**. Cuando estás en la última posición y saltas, vuelves al inicio. Esto es un patrón común que verás en muchos problemas.

PASO 5: Verificar

Caso normal: $c = [0, 0, 1, 0, 0, 1, 1, 0]$, $k = 2$, $e = 100 \rightarrow 92 \checkmark$

Edge case — sin truenos: $c = [0, 0, 0, 0]$, $k = 1$, $e = 10 \rightarrow 4$ saltos, sin truenos $\rightarrow 10 - 4 = 6 \checkmark$

Edge case — todas trueno: $c = [1, 1, 1, 1]$, $k = 2$, $e = 10 \rightarrow 2$ saltos, 2 truenos $\rightarrow 10 - 2 - 4 = 4 \checkmark$

Lección de Jumping on Clouds

Concepto	Lo que aprendimos
n pequeño	Cuando $n \leq 25-50$, casi cualquier complejidad funciona
Simular es válido	Si las constraints lo permiten, simular paso a paso está bien
Módulo para circular	<code>% c.length</code> crea un array circular (el último salta al primero)

Concepto	Lo que aprendimos
$O(n/k)$	Un loop con salto de k tiene complejidad $O(n/k)$, no $O(n)$

Ejemplo 3: Circular Array Rotation

Problema en HackerRank

El problema

Dado un array de enteros, rota el array k veces a la derecha. Después, responde q queries: "¿qué valor hay en el índice m ?"

Rotación a la derecha: el último elemento se mueve al inicio, todos los demás se corren a la derecha.

Ejemplo: $a = [1, 2, 3]$, $k = 2$ rotaciones:

- Rotación 1: $[3, 1, 2]$
- Rotación 2: $[2, 3, 1]$
- Query índice $0 \rightarrow 2$, índice $1 \rightarrow 3$, índice $2 \rightarrow 1$

PASO 1: Entender el problema

1. **¿Qué me dan?** Un array, un número de rotaciones k , y una lista de queries (índices).
2. **¿Qué me piden?** Los valores en los índices dados **después** de rotar k veces.
3. **¿Qué es rotar?** Mover todos los elementos a la derecha, el último pasa al inicio.

PASO 2: Descifrar las constraints

$1 \leq n \leq 10^5$
 $1 \leq k \leq 10^5$
 $1 \leq q \leq 500$

Aquí hay **dos números grandes**: n y k , ambos hasta 100,000.

Opción 1: Simular las rotaciones. Cada rotación recorre todo el array ($O(n)$) y haces k rotaciones $\rightarrow O(n \times k)$. Con $n = 100,000$ y $k = 100,000 \rightarrow 10$ mil millones de operaciones. **No cabe.**

Opción 2: Buscar una fórmula. Si puedo calcular la posición final sin simular, puedo hacerlo en $O(n)$ o mejor.

Necesito $O(n)$ o $O(n + q)$.

PASO 3: Analizar mi primera idea (brute force)

La idea obvia: **simular cada rotación**.

```
// Brute force: O(n × k) — DEMASIADO LENTO
List<int> circularArrayRotation(List<int> a, int k, List<int> queries) {
    for (int r = 0; r < k; r++) {          // ← k rotaciones
        int last = a.last();
        a.removeLast();                    // O(n) — desplaza elementos
        a.insert(0, last);                  // O(n) — desplaza elementos
    }

    List<int> result = [];
    for (int q in queries) {
        result.add(a[q]);
    }
    return result;
}
```

Análisis:

1. El primer loop se ejecuta k veces.
2. Dentro: `removeLast()` es $O(1)$, pero `insert(0, last)` es $O(n)$ porque mueve todos los elementos.
3. Total: $k \times n$ operaciones.

$O(n \times k)$ — Con $n = 100,000$ y $k = 100,000 = 10^{10}$ operaciones. No cabe.

PASO 4: Optimizar — La fórmula mágica

Pregunta clave: ¿Necesito realmente simular las rotaciones?

Si roto un array de tamaño n , k veces, el elemento que estaba en la posición i termina en la posición $(i + k) \% n$.

Ejemplo: Array `[1, 2, 3, 4, 5]`, $k = 2$ rotaciones.

Posición original	Posición después de 2 rotaciones
$0 \rightarrow (0+2) \% 5 = 2$	El 1 queda en la posición 2
$1 \rightarrow (1+2) \% 5 = 3$	El 2 queda en la posición 3
$2 \rightarrow (2+2) \% 5 = 4$	El 3 queda en la posición 4
$3 \rightarrow (3+2) \% 5 = 0$	El 4 queda en la posición 0
$4 \rightarrow (4+2) \% 5 = 1$	El 5 queda en la posición 1

Resultado: `[4, 5, 1, 2, 3]` ✓

Pero no necesito construir el array rotado. Si me preguntan por el índice m , puedo calcular directamente: "¿Qué elemento original terminó en la posición m ?"

La fórmula inversa: si el elemento original en posición i va a parar a $(i + k) \% n$, entonces el elemento en la posición m después de rotar es el que estaba en la posición $(m - k \% n + n) \% n$ original.

```
// Optimizado: O(n + q) — construir mapa de respuestas
List<int> circularArrayRotation(List<int> a, int k, List<int> queries) {
    int n = a.length;
    List<int> result = [];

    for (int q in queries) {
        // ¿Qué elemento original está en la posición q después de k rotaciones?
        int originalIndex = (q - k % n + n) % n;
        result.add(a[originalIndex]);
    }
    return result;
}
```

Análisis:

1. Un loop sobre las queries: q iteraciones.
2. Dentro: operaciones aritméticas $O(1)$.
3. No construyo el array rotado.

Complejidad: $O(q)$ — donde q es el número de queries. Con $q \leq 500$, esto es instantáneo.

¿Por qué funciona? Porque la rotación es una operación **aritmética**, no necesito simularla. Es como saber que girar un reloj 13 horas es lo mismo que girar 1 hora ($13 \% 12 = 1$).

PASO 5: Verificar

Caso normal: $a = [1, 2, 3]$, $k = 2$, queries = $[0, 1, 2] \rightarrow [2, 3, 1]$ ✓

Edge case — $k > n$: $a = [1, 2, 3]$, $k = 5 \rightarrow 5 \% 3 = 2$ rotaciones efectivas $\rightarrow [2, 3, 1]$ ✓

Edge case — $k = 0$: $a = [1, 2, 3]$, $k = 0 \rightarrow$ sin rotación $\rightarrow [1, 2, 3]$ ✓

Lección de Circular Array Rotation

Concepto	Lo que aprendimos
No siempre simular	A veces una fórmula matemática reemplaza un loop pesado
Módulo para circular	$\% n$ convierte posiciones lineales en circulares
$O(n \times k) \rightarrow O(q)$	La optimización puede ser dramática
Identificar la fórmula	Pregúntate: "¿Puedo calcular la respuesta sin construir todo?"

Ejemplo 4: Sequence Equation

Problema en HackerRank

El problema

Dada una secuencia p de n enteros distintos donde cada elemento satisface $1 \leq p[i] \leq n$, para cada x de 1 a n , encuentra un y tal que $p(p(y)) = x$.

Ejemplo: $p = [2, 3, 1]$ (donde $p[1]=2$, $p[2]=3$, $p[3]=1$)

- $x = 1$: necesito y donde $p(p(y)) = 1$. Si $y = 2$, $p(2) = 3$, $p(3) = 1 \checkmark \rightarrow y = 2$
- $x = 2$: necesito y donde $p(p(y)) = 2$. Si $y = 1$, $p(1) = 2$, $p(2) = 3 \dots$ no. Si $y = 3$, $p(3) = 1$, $p(1) = 2 \checkmark \rightarrow y = 3$
- $x = 3$: necesito y donde $p(p(y)) = 3$. Si $y = 1$, $p(1) = 2$, $p(2) = 3 \checkmark \rightarrow y = 1$
- Respuesta: $[2, 3, 1]$

PASO 1: Entender el problema

1. **¿Qué me dan?** Una permutación p (números del 1 al n , todos distintos).
2. **¿Qué me piden?** Para cada x , encontrar y tal que $p(p(y)) = x$.
3. **¿Qué significa $p(p(y)) = x$?** Primero aplico p a y , luego aplico p al resultado, y debe dar x .

Reformulación: Si p es una función, busco y tal que $p(p(y)) = x$. Es como "aplicar p dos veces".

PASO 2: Descifrar las constraints

```
1 ≤ n ≤ 50
1 ≤ p[i] ≤ n
```

$n \leq 50$. Esto es **muy pequeño**. Cualquier complejidad funciona. Pero vamos a practicar el análisis.

PASO 3: Analizar mi primera idea (brute force)

La idea: **probar cada y posible** para cada x .

```
// Brute force: O(n³) – funciona pero es lento
List<int> permutationEquation(List<int> p) {
    List<int> result = [];
    int n = p.length;

    for (int x = 1; x <= n; x++) {          // Para cada x
        for (int y = 1; y <= n; y++) {      // Probar cada y
            if (p[p[y] - 1] - 1 == x) {      // ¿p(p(y)) = x?
                result.add(y);
                break;
            }
        }
    }
    return result;
}
```

Análisis:

1. Loop externo: n iteraciones (para cada x).
2. Loop interno: n iteraciones (probar cada y).
3. Dentro: una operación de array $O(1)$.

$O(n^2)$ — Con $n = 50$, son 2,500 operaciones. Cabe perfectamente.

Pero podemos hacerlo mejor.

PASO 4: Optimizar — Pre-computar con Map

Pregunta clave: ¿Puedo calcular la respuesta sin probar cada y ?

Observación: Si $p(p(y)) = x$, entonces $p(y) = p^{-1}(x)$ (el inverso de p aplicado a x). Es decir:

1. Primero invierto p : creo un mapa donde $inverse[p[i]] = i + 1$.
2. Luego, para cada x , $y = inverse[inverse[x]]$.

```
// Optimizado:  $O(n)$  con pre-computación
List<int> permutationEquation(List<int> p) {
    int n = p.length;

    // Paso 1: Construir el mapa inverso —  $O(n)$ 
    Map<int, int> inverse = {};
    for (int i = 0; i < n; i++) {
        inverse[p[i]] = i + 1; //  $p[i]$  = valor → posición
    }

    // Paso 2: Para cada  $x$ ,  $y = inverse[inverse[x]]$  —  $O(n)$ 
    List<int> result = [];
    for (int x = 1; x <= n; x++) {
        result.add(inverse[inverse[x]]);
    }
    return result;
}
```

Análisis:

1. Construir el mapa inverso: un loop de n iteraciones $\rightarrow O(n)$.
2. Construir el resultado: otro loop de n iteraciones $\rightarrow O(n)$.
3. Dentro de cada loop: operaciones $O(1)$ en el Map.

$O(n) + O(n) = O(n)$ — Complejidad lineal.

¿Por qué funciona? Porque el mapa inverso me permite "deshacer" la función p en $O(1)$. Es como tener un diccionario: en vez de probar todas las traducciones, busco directamente.

PASO 5: Verificar

Caso normal: $p = [2, 3, 1] \rightarrow [2, 3, 1]$ ✓

Caso 2: $p = [4, 3, 5, 1, 2] \rightarrow [1, 3, 5, 4, 2]$ ✓

Edge case — $n = 1$: `p = [1]` → `inverse = {1: 1}` → `y = inverse[inverse[1]] = inverse[1] = 1` → `[1]` ✓

Lección de Sequence Equation

Concepto	Lo que aprendimos
Pre-computar	Construir un mapa auxiliar responde queries en $O(1)$
Map inverso	Invertir una función: <code>f(x) = y</code> → <code>inverse[y] = x</code>
$O(n^2) \rightarrow O(n)$	Pre-computar transforma un problema cuadrático en lineal
Patrón de pre-computación	Si vas a preguntar lo mismo muchas veces, pre-calcula

PARTE C: Referencia rápida

Después de resolver problemas, usa esta sección como **consulta rápida**. Aquí están los conceptos teóricos organizados para que los busques cuando los necesites.

C.1 Las 4 complejidades esenciales

Estas son las que necesitas **memorizar**. Las demás son casos extremos.

$O(1)$ — Constante

Qué significa: El tiempo **no cambia** sin importar cuántos datos tengas.

Analogía:

- Buscar tu nombre en la guía telefónica si sabes que empieza con "M" — vas directo a esa sección.
- Abrir un libro en una página marcada con un post-it.

En código:

```
int obtenerPrimero(List<int> arr) {  
    return arr[0]; // Siempre 1 operación, sin importar el tamaño  
}
```

Ejemplos cotidianos:

- Acceder al primer elemento de un array: `arr[0]`
- Buscar en un HashMap: `map[key]`
- Sumar dos números: `a + b`

Cuándo la ves: Operaciones que **no dependen del tamaño** de los datos.

O(n) — Lineal

Qué significa: El tiempo crece **en la misma proporción** que los datos.

Analogía:

- Revisar un listado completo de tareas — lees cada tarea una por una.
- Leer un libro página por página.

En código:

```
int sumarTodo(List<int> arr) {  
    int total = 0;  
    for (int x in arr) {    // Recorre cada elemento una vez  
        total += x;  
    }  
    return total;  
}
```

Ejemplos numéricos:

- 10 datos → 10 operaciones
- 100 datos → 100 operaciones
- 1,000 datos → 1,000 operaciones

Cuándo la ves: Recorrer un array, contar frecuencias, buscar en lista no ordenada.

O(n²) — Cuadrático

Qué significa: El tiempo crece **al cuadrado** de los datos.

Analogía:

- Comparar cada libro de una estantería con todos los demás.
- En una clase, que cada alumno compare su cuaderno con el de todos los demás.

En código:

```
bool tieneDuplicados(List<int> arr) {  
    for (int i = 0; i < arr.length; i++) {  
        for (int j = i + 1; j < arr.length; j++) {    // Para cada i, recorre todo lo que queda  
            if (arr[i] == arr[j]) return true;  
        }  
    }  
    return false;  
}
```

Ejemplos numéricos:

- 10 datos → ~50 operaciones (10 × 5)
- 100 datos → ~5,000 operaciones (100 × 50)
- 1,000 datos → ~500,000 operaciones

⚠ **Problema:** Con $n = 10,000$, ya son 100 millones de operaciones. Con $n = 100,000$, serían 10 mil millones — demasiado para 1 segundo.

Revisual: Cuando ves `for` + `for` anidados que recorren los mismos datos, es $O(n^2)$.

$O(\log n)$ — Logarítmica

Qué significa: El tiempo crece **muy lento**. Cada vez que duplicas los datos, solo necesitas **1 paso más**.

Analogía:

- Buscar una palabra en el diccionario: abres por la mitad, decides si ir a la izquierda o derecha, y repites.
- Adivinar un número entre 1 y 100: preguntas "¿es mayor que 50?" → si es sí, buscas en 51-100; si es no, buscas en 1-49.

En código:

```
int busquedaBinaria(List<int> arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) ~/ 2; // Divides por la mitad

        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

¿Por qué $\log n$? Porque $\log_2(n) = \text{"¿cuántas veces tengo que dividir } n \text{ por } 2 \text{ para llegar a } 1\text{"}$

Tabla de ejemplos: | Datos (n) | Pasos necesarios ($\log_2(n)$) | |-----|-----| | 8 | 3 (8 → 4 → 2 → 1) | | 1,000 | ~10 | | 1,000,000 | ~20 | | 1,000,000,000 | ~30 |

La clave: Con 1,000,000 de datos, solo necesitas ~20 pasos. ¡Increíblemente eficiente!

C.2 Tabla de crecimiento visual

Cuántas operaciones hace cada complejidad según el tamaño de los datos:

Complejidad	$n = 10$	$n = 100$	$n = 1,000$	$n = 10,000$
$O(1)$	1	1	1	1
$O(\log n)$	3	7	10	13
$O(n)$	10	100	1,000	10,000
$O(n \log n)$	30	700	10,000	130,000
$O(n^2)$	100	10,000	1,000,000	100,000,000

- **O(1)** siempre es 1 operación.
- **O(log n)** crece muy lento. De 10 a 10,000 datos (1000× más), solo pasas de 3 a 13 operaciones (4× más).
- **O(n)** crece proporcionalmente. 100× más datos = 100× más tiempo.
- **O(n²)** crece explosivamente. 100× más datos = 10,000× más tiempo.

C.3 Complejidades que debes conocer (pero no memorizar)

Complejidad	Cuándo aparece	¿Es aceptable?
O(n log n)	Sorting (merge sort, quicksort)	Sí, para $n \leq 10^5$
O(n³)	Tres loops anidados, Floyd-Warshall	Solo para $n \leq 500$
O(2ⁿ)	Probar todas las combinaciones	Solo para $n \leq 20$
O(n!)	Permutaciones de todos los elementos	Solo para $n \leq 10$

C.4 Cómo analizar código: 3 patrones

No necesitas memorizar reglas complejas. Solo necesitas reconocer **3 patrones**. Aquí te enseño cómo identificarlos en código real.

Patrón 1: Un solo bucle → O(n)

Cómo reconocerlo: Un solo `for` o `while` que recorre los datos una vez.

Ejemplo simple:

```
int sumar(List<int> arr) {
    int total = 0;
    for (int x in arr) { // ← Un solo loop que recorre todo
        total += x;
    }
    return total;
}
```

Análisis paso a paso:

1. ¿Cuántos loops hay? Solo 1
2. ¿Cuántas veces se ejecuta? `arr.length` veces (n veces)
3. ¿Qué hay dentro? Una suma (operación O(1))
4. **Total:** $n \times 1 = n$ operaciones → **O(n)**

Señal para buscar: `for (cada elemento) → hacer algo`

Otros ejemplos de O(n):

- Contar frecuencias de un elemento
 - Buscar el máximo/mínimo en un array
 - Copiar un array a otro
-

Patrón 2: Bucles anidados → $O(n^2)$

Cómo reconocerlo: Dos `for` o `while` anidados que recorren los mismos datos.

Ejemplo simple:

```
bool hayDuplicado(List<int> arr) {  
    for (int i = 0; i < arr.length; i++) {           // ← Primer loop: n veces  
        for (int j = i + 1; j < arr.length; j++) {    // ← Segundo loop: n veces  
            if (arr[i] == arr[j]) return true;  
        }  
    }  
    return false;  
}
```

Análisis paso a paso:

1. **¿Cuántos loops hay?** 2 (anidados)
2. **¿Cuántas veces se ejecuta cada uno?** Ambos n veces
3. **Total:** $n \times n = n^2$ operaciones → **$O(n^2)$**

Señal para buscar: `for (cada elemento) { for (cada otro elemento) }`

¿Por qué es n^2 ? Porque para cada elemento del array, recorres **todos** los demás.

Variante importante: Si el segundo loop empieza en `i` (no en 0), sigue siendo $O(n^2)$. Big-O ignora constantes como 1/2.

Otros ejemplos de $O(n^2)$:

- Encontrar todos los pares de un array
 - Ordenar con bubble sort
 - Comparar cada elemento con todos los demás
-

Patrón 3: Dividir por la mitad → $O(\log n)$

Cómo reconocerlo: Un `while` que en cada paso divide el espacio de búsqueda por 2.

Ejemplo simple:

```
int busquedaBinaria(List<int> arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) { // ← Se ejecuta  $\log_2(n)$  veces
        int mid = left + (right - left) ~/ 2; // Divides por la mitad
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Análisis paso a paso:

1. **¿Qué hace el loop?** Divide el rango por 2 en cada paso
2. **¿Cuántas veces se ejecuta?** Hasta que $\text{left} > \text{right}$
3. **¿Cuántos pasos para n elementos?** $\log_2(n)$ veces
4. **Total:** $\log_2(n)$ operaciones $\rightarrow O(\log n)$

Señal para buscar: `while (n > 1) { n = n ~/ 2; }`

Pregunta clave: "¿En cada paso, descarto la mitad de los datos?" Si es sí, es $O(\log n)$.

Tabla de ejemplos:

Datos (n)	Pasos ($\log_2(n)$)
16	4 ($16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$)
1,000	~ 10
1,000,000	~ 20

¿Cómo identificar el patrón en código nuevo?

Paso 1: Busca `for` o `while` en el código **Paso 2:** Cuenta cuántos hay y si están anidados **Paso 3:** Pregúntate:

- Si hay **1 loop** $\rightarrow$ probablemente $O(n)$
- Si hay **2 loops anidados** $\rightarrow$ probablemente $O(n^2)$
- Si hay **un while que divide por 2** $\rightarrow$ probablemente $O(\log n)$

Ejercicio rápido: ¿Qué complejidad tiene este código?

```
for (int i = 0; i < n; i++) {
    print(arr[i]);
}
for (int j = 0; j < n; j++) {
    print(arr[j]);
}
```

Respuesta: $O(n) + O(n) = O(n)$ — son dos loops secuenciales, no anidados.

Mapa de decisiones: ¿Qué complejidad tiene mi código?

Cuando no sabes por dónde empezar, usa este mapa:

```
¿Tu código tiene loops?
├─ NO →  $O(1)$  – probablemente constante
├─ Sí, 1 solo loop
│   ├── ¿Recorre todo el array? →  $O(n)$ 
│   ├── ¿Divide por la mitad en cada paso? →  $O(\log n)$ 
│   └── ¿Salta de k en k posiciones? →  $O(n/k) \approx O(n)$ 
├─ Sí, 2 loops anidados
│   ├── ¿Ambos recorren el mismo array? →  $O(n^2)$ 
│   └── ¿Recorren arrays de tamaño diferente? →  $O(n \times m)$ 
├─ Sí, 3+ loops anidados
│   └──  $O(n^3)$  o peor – busca optimizar
└─ Sí, pero son secuenciales (uno tras otro)
    └── Suma las complejidades:  $O(n) + O(n^2) = O(n^2)$ 
```

Cómo usarlo:

1. Mira tu código
2. Responde las preguntas del mapa
3. Llega a tu complejidad
4. Compara con la tabla de constraints (sección PASO 2)

C.5 Reglas de combinación

En la vida real, los algoritmos combinan patrones. Aquí tienes las reglas para combinar complejidades.

Regla 1: Operaciones secuenciales → toma la MÁS GRANDE

Cuándo la ves: Cuando tienes dos o más partes de código que se ejecutan una después de otra (no anidadas).

Ejemplo:

```
void ejemplo(List<int> arr) {
    // Parte 1:  $O(n)$  – un solo loop
    for (int x in arr) {
        print(x);
    }

    // Parte 2:  $O(n^2)$  – dos loops anidados
    for (int i = 0; i < arr.length; i++)
        for (int j = 0; j < arr.length; j++) {
            print(arr[i] + arr[j]);
        }
}
```

Análisis:

1. Parte 1: $O(n)$
2. Parte 2: $O(n^2)$
3. **Total:** $O(n) + O(n^2) = O(n^2)$ — la parte más lenta domina

¿Por qué? Porque $O(n^2)$ es mucho más lento que $O(n)$. Cuando n crece, $O(n^2)$ domina el tiempo total.

Analogía: Si vas al supermercado y tardas 10 minutos en comprar + 1 hora en cocinar, el tiempo total es ~1 hora (el paso más lento domina).

Regla 2: Bucles anidados → se multiplican

Cuándo la ves: Cuando un loop está dentro de otro y dependen de tamaños diferentes.

Ejemplo:

```
for (int i = 0; i < n; i++)    // n veces
  for (int j = 0; j < m; j++)  // m veces
    print(i + j);
```

Análisis:

1. Primer loop: n veces
2. Segundo loop: m veces
3. **Total:** $O(n \times m)$

Caso especial: Si $n = m$ (ambos son el mismo tamaño), entonces $O(n \times n) = O(n^2)$.

Ejemplos prácticos

Ejemplo 1: ¿Qué complejidad tiene?

```
void ejemplo1(List<int> arr1, List<int> arr2) {
  for (int x in arr1) { // O(n) donde n = arr1.length
    print(x);
  }
  for (int y in arr2) { // O(m) donde m = arr2.length
    print(y);
  }
}
```

Respuesta: $O(n) + O(m) = O(n + m)$

Ejemplo 2: ¿Qué complejidad tiene?

```
void ejemplo2(List<int> arr) {
  for (int i = 0; i < arr.length; i++) {
    for (int j = 0; j < arr.length; j++) {
      print(arr[i] + arr[j]);
    }
  }
  print("Terminado");
}
```

Respuesta: $O(n^2) + O(1) = O(n^2)$

Resumen de reglas

Situación	Regla	Ejemplo
Secuencial (uno tras otro)	Toma la MÁS GRANDE	$O(n) + O(n^2) = O(n^2)$
Anidado (uno dentro del otro)	Se MULTIPLICAN	$O(n) \times O(m) = O(n \times m)$
Constante dentro de loop	No cambia	$O(n) \times O(1) = O(n)$

C.6 Complejidad espacial

No solo importa el tiempo. El **espacio** (memoria) también cuenta.

Estructura	Espacio	Cuándo importa
Array de tamaño n	$O(n)$	Siempre es $O(n)$ para el input
HashMap con n elementos	$O(n)$	Cuando el input es grande
Recursión profunda	$O(n)$ stack	Puede causar stack overflow
Variable auxiliar	$O(1)$	El óptimo en espacio

Ejemplo: recursión vs iterativo

```
// O(n) espacio – recursión
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1); // n llamadas apiladas
}

// O(1) espacio – iterativo
int factorialIterativo(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

C.6.1 Cómo analizar complejidad de recursión

La recursión es un caso especial. No puedes simplemente "contar loops" porque la función se llama a sí misma.

La regla: Cuenta cuántas veces se llama la función y cuánto trabajo hace en cada llamada.

Ejemplo 1: Factorial (lineal)

```
int factorial(int n) {
    if (n <= 1) return 1;           // 0(1) – caso base
    return n * factorial(n - 1);    // 1 multiplicación 0(1) + llamada recursiva
}
```

Análisis paso a paso:

Paso	Pregunta	Respuesta
1	¿Cuántas llamadas se hacen?	n llamadas (desde n hasta 1)
2	¿Cuánto trabajo en cada llamada?	1 comparación + 1 multiplicación = O(1)
3	Total tiempo	n × O(1) = O(n)
4	Total espacio	O(n) — cada llamada se apila en el stack

Visualización:

```
factorial(5)
→ 5 * factorial(4)
→ 4 * factorial(3)
→ 3 * factorial(2)
→ 2 * factorial(1)
→ 1 (caso base)
```

Son 5 llamadas apiladas → O(n) espacio.

Ejemplo 2: Fibonacci (exponencial — ¡cuidado!)

```
int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2); // 2 llamadas
}
```

Análisis:

Paso	Pregunta	Respuesta
1	¿Cuántas llamadas por nivel?	Cada llamada genera 2 llamadas más
2	¿Cuántos niveles?	n niveles
3	Total	2 ⁿ llamadas → O(2ⁿ) — ¡muy lento!

¿Por qué es tan lento? Porque se repite el mismo cálculo muchas veces:

```
fibonacci(5)
  → fibonacci(4) + fibonacci(3)
    → fibonacci(3) + fibonacci(2) + fibonacci(3) se calcula DOS veces
      → fibonacci(2) + fibonacci(1) + fibonacci(2) se calcula TRES veces
```

Ejemplo 3: Búsqueda binaria recursiva (logarítmica)

```
int busquedaBinaria(List<int> arr, int target, int left, int right) {
    if (left > right) return -1;
    int mid = left + (right - left) ~/ 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target)
        return busquedaBinaria(arr, target, mid + 1, right); // 1 llamada
    else
        return busquedaBinaria(arr, target, left, mid - 1); // 1 llamada
}
```

Análisis:

Paso	Pregunta	Respuesta
1	¿Cuántas llamadas?	$\log_2(n)$ — cada llamada descarta la mitad
2	¿Trabajo en cada llamada?	$O(1)$
3	Total	$O(\log n)$

Reglas para recursión

Si cada llamada hace...	Complejidad	Ejemplo
1 llamada recursiva con $n-1$	$O(n)$	Factorial
1 llamada recursiva con $n/2$	$O(\log n)$	Búsqueda binaria
2 llamadas recursivas	$O(2^n)$	Fibonacci malo
2 llamadas recursivas + ordenar	$O(n \log n)$	Mergesort

Truco: Si el código recursivo se parece a un loop (1 llamada, reduce n en cada paso), probablemente es $O(n)$ o $O(\log n)$. Si se parece a un árbol (2+ llamadas), probablemente es $O(2^n)$ o peor.

C.7 El Tradeoff Tiempo-Espacio

Muchas veces puedes intercambiar tiempo por espacio:

Enfoque	Tiempo	Espacio
Brute force (re-calculando todo)	$O(n^2)$	$O(1)$

Enfoque	Tiempo	Espacio
Pre-computar con HashMap	$O(n)$	$O(n)$
Pre-computar con Prefix Sum	$O(n)$	$O(n)$

Ejemplo: Quieres saber la suma de cada subarray.

- **Brute force:** Para cada subarray, recorres todos los elementos $\rightarrow O(n^2)$
- **Prefix Sum:** Pre-calculas las sumas acumuladas $\rightarrow O(n)$ pero usas $O(n)$ de espacio extra

Regla general: Si las constraints de memoria son amplias (típico en entrevistas), prioriza tiempo.

C.8 Complejidad de cada estructura de datos

Estructura	Acceso	Búsqueda	Inserción	Eliminación
List (Array)	$O(1)$ por índice	$O(n)$	$O(1)$ amortizado (final), $O(n)$ (medio)	$O(n)$ (medio)
Set	—	$O(1)$	$O(1)$	$O(1)$
Map (HashMap)	$O(1)$ por key	$O(1)$	$O(1)$	$O(1)$
Queue	$O(1)$ primero	$O(n)$	$O(1)$ addLast	$O(1)$ removeFirst
PriorityQueue (Heap)	$O(1)$ min/max	$O(n)$	$O(\log n)$	$O(\log n)$

¿Qué significa esto para ti? Si necesitas buscar algo repetidamente, usa **Set** o **Map** ($O(1)$) en vez de **List** ($O(n)$).

Checklist antes de codificar

- ☐ ¿Cuál es el tamaño máximo de n ?
- ☐ ¿Qué complejidad necesito según la tabla?
- ☐ ¿Mi enfoque cumple esa complejidad?
- ☐ ¿Puedo usar un Map/Set para hacer búsquedas $O(1)$?
- ☐ ¿Necesito pre-computar algo?
- ☐ ¿Cuánto espacio extra uso?

Ejercicios prácticos: Analiza la complejidad

Para cada fragmento de código, sigue el checklist y determina la complejidad. Intenta resolverlos antes de ver la respuesta.

Ejercicio 1: Dos loops secuenciales

```
void ejemplo(List<int> arr) {  
    for (int i = 0; i < arr.length; i++) {  
        print(arr[i]);  
    }  
    for (int j = 0; j < arr.length; j++) {  
        print(arr[j]);  
    }  
}
```

► Ver análisis

Ejercicio 2: Loop con búsqueda en Array

```
bool tieneDoble(List<int> arr) {  
    for (int i = 0; i < arr.length; i++) {  
        if (arr.contains(arr[i] * 2)) { // ← contains es O(n)  
            return true;  
        }  
    }  
    return false;  
}
```

► Ver análisis

Ejercicio 3: Loop con búsqueda en Map

```
bool tieneRepetido(List<int> arr) {  
    Map<int, bool> seen = {};  
    for (int i = 0; i < arr.length; i++) {  
        if (seen.containsKey(arr[i])) { // ← containsKey es O(1)  
            return true;  
        }  
        seen[arr[i]] = true;  
    }  
    return false;  
}
```

► Ver análisis

Ejercicio 4: Recursión

```
int suma(int n) {  
    if (n <= 0) return 0;  
    return n + suma(n - 1);  
}
```

► Ver análisis

Ejercicio 5: Dos loops con tamaños diferentes

```
void ejemplo(List<int> arr1, List<int> arr2) {  
    for (int x in arr1) {  
        print(x);  
    }  
    for (int y in arr2) {  
        print(y);  
    }  
}
```

► Ver análisis

¿Acertaste todos? Si sí, entiendes lo básico de análisis de complejidad. Si no, revisa el checklist de la sección anterior y vuelve a intentarlo.

PARTE D: Preguntas frecuentes de principiantes

Si eres nuevo en Big-O, probablemente tengas algunas de estas preguntas. Aquí las respondo de forma simple.

P1: ¿Qué es exactamente Big-O?

Big-O es una **clasificación** que nos dice cómo crece el tiempo de ejecución de un algoritmo cuando aumentan los datos.

No es una fórmula matemática complicated — es como una **etiqueta** que pones a tu código:

- "Este código es $O(n)$ " = crece linealmente
- "Este código es $O(n^2)$ " = crece exponencialmente

Analogía: Es como clasificar carros por velocidad:

- "Este carro va a 100 km/h" (rápido)
- "Este carro va a 50 km/h" (lento)

P2: ¿Por qué se ignoran las constantes?

Ejemplo:

- Código A: $3n$ operaciones

- Código B: `100n` operaciones

¿Cuál es más rápido? Código A ($3n < 100n$)

¿Pero en Big-O? Ambos son **$O(n)$**

¿Por qué? Porque cuando n crece mucho (ej: 1,000,000), la constante (3 vs 100) no importa tanto como el **patrón de crecimiento**.

n	3n	100n	Diferencia
10	30	1,000	33×
1,000	3,000	100,000	33×
1,000,000	3,000,000	100,000,000	33×

La diferencia siempre es 33×, pero ambos crecen **linealmente**. Por eso decimos que ambos son $O(n)$.

P3: ¿Cómo sé qué es "n" en un problema?

Regla simple: "n" es generalmente el **tamaño del input principal**.

Ejemplos:

- Array de 100 elementos $\rightarrow n = 100$
- String de 50 caracteres $\rightarrow n = 50$
- Rango de días de 20 a 23 $\rightarrow n = 23 - 20 + 1 = 4$

En problemas de HackerRank: Las constraints te dicen cuál es n:

$1 \leq n \leq 10^5 \rightarrow n$ es el tamaño del array

P4: ¿Qué pasa si tengo múltiples inputs con tamaños diferentes?

Ejemplo:

```
void ejemplo(List<int> arr1, List<int> arr2) {  
    for (int x in arr1) { ... } //  $O(n)$  donde  $n = \text{arr1.length}$   
    for (int y in arr2) { ... } //  $O(m)$  donde  $m = \text{arr2.length}$   
}
```

Complejidad total: $O(n + m)$

Regla: Si los inputs son independientes, usa letras diferentes (n, m, p, etc.)

P5: ¿ $O(n^2)$ es siempre malo?

No siempre. Depende de las constraints.

Si n es...	$O(n^2)$ es...	Ejemplo
10	100 ops → acceptable	Problemas pequeños
100	10,000 ops → acceptable	Problemas medianos
1,000	1,000,000 ops → acceptable	Problemas grandes
10,000	100,000,000 ops → límite	¡Cuidado!
100,000	10,000,000,000 ops → malo	Time Limit Exceeded

Conclusión: $O(n^2)$ puede ser aceptable si n es pequeño.

P6: ¿Cómo mejoro de $O(n^2)$ a $O(n)$?

Técnica común: Usar **HashMap** (Map en Dart) para búsquedas $O(1)$.

Antes ($O(n^2)$):

```
for (int i = 0; i < n; i++) {  
  for (int j = 0; j < n; j++) { // Búsqueda  $O(n)$   
    if (arr[i] + arr[j] == target) return [i, j];  
  }  
}
```

Después ($O(n)$):

```
Map<int, int> seen = {};  
for (int i = 0; i < n; i++) {  
  int complemento = target - arr[i];  
  if (seen.containsKey(complemento)) return [seen[complemento]!, i]; // Búsqueda  $O(1)$   
  seen[arr[i]] = i;  
}
```

La clave: El HashMap convierte una búsqueda $O(n)$ en $O(1)$.

P7: ¿Qué es la complejidad espacial?

Complejidad temporal: Cuánto **tiempo** tarda tu algoritmo. **Complejidad espacial:** Cuánta **memoria** usa tu algoritmo.

Ejemplo:

```
// O(n) espacio – crea un array nuevo
List<int> duplicar(List<int> arr) {
    List<int> nuevo = [];
    for (int x in arr) {
        nuevo.add(x * 2);
    }
    return nuevo;
}

// O(1) espacio – modifica el array original
void duplicarEnLugar(List<int> arr) {
    for (int i = 0; i < arr.length; i++) {
        arr[i] *= 2;
    }
}
```

¿Cuándo importa? Cuando el input es muy grande y la memoria es limitada.

P8: ¿Cómo practico para entender mejor Big-O?

Ejercicio diario:

1. Toma un código que escribiste
2. Pregúntate: "¿Cuántos loops hay? ¿Están anidados?"
3. Clasifica: $O(1)$, $O(n)$, $O(n^2)$, o $O(\log n)$
4. Verifica con restricciones del problema

Recursos para practicar:

- HackerRank (sección "Algorithms")
- LeetCode (problemas "Easy")
- Exercism (tracks de Dart/Flutter)

P9: ¿Big-O es lo mismo que "eficiencia"?

No exactamente. Big-O mide **crecimiento**, no eficiencia absoluta.

Ejemplo:

- Algoritmo A: $O(n^2)$ pero con operaciones simples
- Algoritmo B: $O(n)$ pero con operaciones complejas

Para n pequeño: Algoritmo A podría ser más rápido **Para n grande:** Algoritmo B será más rápido

Regla práctica: Primero optimiza la complejidad (Big-O), luego optimiza las constantes.

P10: ¿Qué hago si no sé la complejidad de mi código?

Sigue estos pasos:

1. **Cuenta los loops** (¿cuántos hay? ¿están anidados?)
2. **Identifica operaciones costosas** (búsquedas en arrays, recursión)
3. **Usa los patrones** de la sección C.4
4. **Compara con restricciones** del problema

Si todavía no sabes: Ejecuta tu código con inputs de diferentes tamaños y mide el tiempo. Si el tiempo se duplica cuando n se duplica $\rightarrow O(n)$. Si se cuadruplica $\rightarrow O(n^2)$.

03 — Reconocimiento de Patrones

El 80% de los problemas de algoritmos caen en ~15 patrones. Tu trabajo no es inventar un algoritmo desde cero, sino **reconocer** cuál patrón aplica.

Framework de 3 pasos para reconocer el patrón

1. Identificar tipo de input
2. Detectar keywords del enunciado
3. Validar contra las constraints

Paso 1: Tipo de Input → Patrones candidatos

Tipo de Input	Patrones a considerar primero
Array (ordenado)	Two Pointers, Binary Search
Array (desordenado)	HashMap, Sorting + Two Pointers, Sliding Window
String	HashMap (frecuencias), Sliding Window, Trie
Linked List	Two Pointers (fast/slow), Reversal
Tree	DFS, BFS, Recursión
Graph	BFS, DFS, Topological Sort, Union-Find
Matrix / Grid	BFS + visited, DFS, DP
Intervalos	Sorting + Greedy, Merge logic
Stream / Online	Heap / Priority Queue
Números / Bits	Bit manipulation, Math, Binary Search

Paso 2: Keywords del Problema → Patrón

Keyword / Frase en el enunciado	Patrón	Ejemplo de problema
"subarray/substring contiguo "	Sliding Window	Máximo subarray de tamaño K
"longest / shortest subarray con..."	Sliding Window	Longest Substring Without Repeating
" par / triplet que suma..."	Two Pointers o HashMap	Two Sum, 3Sum
" sorted array"	Binary Search o Two Pointers	Search in Rotated Array
" minimum steps / shortest path" (sin peso)	BFS	Castle on the Grid
"all combinations / permutations"	Backtracking	Subsets, Permutations
" k-th largest / smallest"	Heap / Priority Queue	Kth Largest Element
" overlapping subproblems + optimal"	Dynamic Programming	Climbing Stairs, Knapsack
" ciclo / cycle"	DFS + visited o Union-Find	Detect Cycle in Linked List
" connected / componentes"	DFS / BFS / Union-Find	Number of Islands
" next greater / smaller element"	Monotonic Stack	Daily Temperatures
" dependency / prerequisite"	Topological Sort	Course Schedule
"words starting with prefix "	Trie	Implement Trie
" merge intervals / overlapping"	Sorting + Intervals	Merge Intervals
" optimal / maximum / minimum" (sin subproblemas)	Greedy	Jump Game
" frequency / count duplicates"	HashMap	Group Anagrams
" all subsets / combinations con restricciones"	Backtracking	Combination Sum

Paso 3: Árbol de Decisión

Responde estas preguntas en orden para llegar al patrón:


```

¿El input es un array/string?
├── Sí
│   ├── ¿Está ordenado?
│   │   ├── Sí → Two Pointers o Binary Search
│   │   └── NO
│   │       ├── ¿Buscas algo contiguo? → Sliding Window
│   │       ├── ¿Buscas un par/suma? → HashMap
│   │       ├── ¿Puedes ordenarlo? → Sorting + Two Pointers
│   │       └── ¿Range sum queries? → Prefix Sum
│   └── ¿Es un stream infinito? → Heap
└── ¿El input es un tree?
    ├── ¿Camino más corto? → BFS
    ├── ¿Explorar todos los caminos? → DFS
    └── ¿BST property? → Binary Search en tree

├── ¿El input es un graph?
    ├── ¿Camino más corto (sin peso)? → BFS
    ├── ¿Explorar/connectivity? → DFS
    ├── ¿Orden de dependencias? → Topological Sort
    ├── ¿Grupos dinámicos? → Union-Find
    └── ¿Camino más corto (con peso)? → Dijkstra

├── ¿El input es un grid?
    ├── ¿Camino más corto? → BFS + visited
    ├── ¿Contar islas/componentes? → DFS + visited
    └── ¿Camino con costo? → DP o Dijkstra

└── ¿El input son números?
    ├── ¿Buscar en rango? → Binary Search
    ├── ¿Optimización? → Binary Search sobre respuesta
    └── ¿Bits/subconjuntos? → Bit Manipulation

```

Tabla Resumen: Los 12 Patrones Esenciales

#	Patrón	Señal principal	Complejidad típica
1	HashMap	"Frequency", "duplicate", "pair with sum"	O(n)
2	Two Pointers	"Sorted", "pair", "triplet", "partition"	O(n)
3	Sliding Window	"Contiguous", "longest/shortest subarray"	O(n)
4	Prefix Sum	"Range sum", "subarray sum = k"	O(n)
5	Binary Search	"Sorted", "min X such that...", monotonic	O(log n)
6	BFS	"Minimum steps", "shortest path" (sin peso)	O(V + E)
7	DFS	"Connected", "all paths", "components"	O(V + E)
8	Greedy	"Optimal", "schedule", locally optimal = globally	O(n log n)
9	DP	"Overlapping subproblems", "optimal value"	Varies
10	Backtracking	"All combinations", "all permutations"	O(2 ⁿ)

#	Patrón	Señal principal	Complejidad típica
11	Heap	"K-th largest", "top K", "median"	$O(n \log k)$
12	Monotonic Stack	"Next greater element", "histogram"	$O(n)$

Cómo Validar tu Elección

Una vez que crees haber identificado el patrón, verifica:

1. **¿La complejidad del patrón cabe en las constraints?** Si el patrón es $O(n^2)$ pero necesitas $O(n)$, no es el correcto.
2. **¿El patrón cubre todos los edge cases?** Si hay negativos, Sliding Window no funciona (usa Prefix Sum).
3. **¿Hay un patrón más simple que funcione?** Si DP y Greedy ambos funcionan, usa Greedy (más simple).

El Mantra

Antes de escribir código, di en voz alta:

"Este es un problema de [PATRÓN] porque el input es [TIPO] y me piden [QUÉ], con complejidad [O(?)]."

Si puedes completar esa frase, ya estás listo para implementar.

04 — Estructuras de Datos: Referencia Rápida

Qué estructura usar para qué situación, con sus complejidades de operación en Dart.

Mapa Rápido

Necesito...	Usa...	Complejidad
Acceso rápido por índice	<code>List</code> (array)	$O(1)$ acceso, $O(n)$ búsqueda
Buscar si un valor existe	<code>Set</code>	$O(1)$ contains
Buscar con frecuencia/pares	<code>Map<K, V></code>	$O(1)$ lookup
Procesar en orden FIFO	<code>Queue</code>	$O(1)$ add/remove
Obtener min/max repetidamente	<code>PriorityQueue</code> (Heap)	$O(\log n)$
Buscar en datos ordenados	Binary Search en <code>List</code>	$O(\log n)$
Conexiones dinámicas	Union-Find (implementación propia)	$O(\alpha(n)) \approx O(1)$
Prefix matching (strings)	Trie (implementación propia)	$O(L)$

Detalle por Estructura

List (Array)

Cuándo usar: Acceso por índice, iteración secuencial, dos pointers, sliding window.

```
// Acceso y actualización
arr[i];           //  $O(1)$ 
arr.add(x);       //  $O(1)$  amortizado
arr.removeAt(i);  //  $O(n)$  – desplaza elementos
arr.sublist(i, j); //  $O(j-i)$ 

// Búsqueda
arr.contains(x);   //  $O(n)$ 
arr.indexOf(x);    //  $O(n)$ 
arr.sort();        //  $O(n \log n)$ 
```

Limitación: Inserción/eliminación en medio es $O(n)$.

Set

Cuándo usar: Verificar existencia, eliminar duplicados, operaciones de conjuntos (unión, intersección).

```
set.contains(x);      // O(1)
set.add(x);           // O(1)
set.remove(x);        // O(1)
set.union(other);     // O(n)
set.intersection(other); // O(min(n,m))
```

Implementación interna: Hash table en Dart.

Map (HashMap)

Cuándo usar: Frecuencias, pares que suman un valor, caching, agrupación.

```
map.containsKey(x);   // O(1)
map[x];               // O(1)
map[x] = value;       // O(1)
map.remove(x);        // O(1)
map.putIfAbsent(x, () => value); // O(1)
map.update(x, (v) => v + 1);    // O(1)
```

Patrón clásico — Frecuencias:

```
Map<String, int> freq = {};
for (var item in list) {
  freq[item] = (freq[item] ?? 0) + 1;
}
```

Queue (Cola)

Cuándo usar: BFS, procesamiento en orden FIFO, sliding window con deque.

```
import 'dart:collection';

Queue<int> q = Queue();
q.addLast(x);    // O(1)
q.removeFirst(); // O(1)
q.first;         // O(1)
q.isEmpty;       // O(1)
```

PriorityQueue (Heap)

Cuándo usar: K-th largest/smallest, merge de K sorted lists, encontrar el elemento más frecuente.

```
import 'package:collection/collection.dart';

// Min-heap por defecto
var heap = PriorityQueue<int>();
heap.add(x);           // O(log n)
heap.removeFirst();    // O(log n) – extrae el mínimo
heap.first;            // O(1)

// Max-heap
var maxHeap = PriorityQueue<int>((a, b) => b.compareTo(a));
```

Patrón Top-K:

```
// K elementos más grandes – usar min-heap de tamaño K
var minHeap = PriorityQueue<int>();
for (var x in list) {
  minHeap.add(x);
  if (minHeap.length > k) {
    minHeap.removeFirst(); // elimina el más pequeño
  }
}
// minHeap contiene los K más grandes
```

Complejidades de Sorting en Dart

Algoritmo	Tiempo	Espacio	Estable	Cuándo
<code>list.sort()</code> (Tim Sort)	$O(n \log n)$	$O(n)$	Sí	Default de Dart
Quick Sort (manual)	$O(n \log n)$ promedio	$O(\log n)$	No	Cuando se necesita in-place
Counting Sort	$O(n + k)$	$O(k)$	Sí	Enteros con rango pequeño

Decisiones de Estructura según el Problema

Problema	Estructura óptima	Por qué
Two Sum	<code>Map<int, int></code>	Lookup de complemento en $O(1)$
Sliding Window Máximo	<code>Deque</code> (Queue doble)	Mantener monotonic, $O(1)$ por operación
K-th Largest	<code>PriorityQueue</code> (min-heap de tamaño k)	$O(n \log k)$ vs $O(n \log n)$ de sorting
Detectar ciclo	Two Pointers (fast/slow)	$O(1)$ espacio, no necesita visited
Union-Find	Implementación propia con path compression	$O(\alpha(n)) \approx O(1)$ amortizado

Problema	Estructura óptima	Por qué
Prefix Sum	<code>List<int></code> precomputada	O(1) por query después de O(n) precompute
Subarray Sum = k	<code>Map<int, int></code> + prefix sum	O(n) con un solo pass

Relación con el Módulo 09

El módulo 09 (Estructuras de Datos con OOP) enseña **cómo implementar y usar** estas estructuras en Dart con ejemplos detallados. Esta sección es una **referencia rápida** para elegir la correcta durante la resolución de problemas.

05 — Patrones Avanzados con Templates en Dart

8 patrones esenciales con template listo para copiar y adaptar. Cada uno incluye: cuándo usar, template Dart, error común, y problema de referencia.

1. Sliding Window

Cuándo usar: Subarray/substring contiguo con una propiedad (longest, shortest, con k elementos).

Template:

```
// Tiempo: O(n) | Espacio: O(1) fijo, O(k) variable
// Sliding Window de tamaño variable
int slidingWindow(List<int> arr, int k) {
  int windowSum = 0;
  int maxSum = 0;

  for (int i = 0; i < arr.length; i++) {
    windowSum += arr[i];

    if (i >= k) {
      windowSum -= arr[i - k]; // shrink desde la izquierda
    }

    if (i >= k - 1) {
      maxSum = max(maxSum, windowSum);
    }
  }
  return maxSum;
}

// Sliding Window de tamaño variable (longest con condición)
int longestWithCondition(List<int> arr) {
  int left = 0;
  int maxLength = 0;
  // state de la ventana (sum, count, etc.)

  for (int right = 0; right < arr.length; right++) {
    // expandir: agregar arr[right] al state

    while (/* condición violada */) {
      // shrink: remover arr[left] del state
      left++;
    }

    maxLength = max(maxLength, right - left + 1);
  }
  return maxLength;
}
```

Error común: Usar Sliding Window cuando hay negativos (rompe la monotonicidad). Usa Prefix Sum en su lugar.

2. Two Pointers

Cuándo usar: Array ordenado, buscar pares/triplets, partition.

Template:

```
// Tiempo: O(n) | Espacio: O(1)
// Two pointers desde ambos extremos
List<int> twoSumSorted(List<int> arr, int target) {
    int left = 0;
    int right = arr.length - 1;

    while (left < right) {
        int sum = arr[left] + arr[right];

        if (sum == target) {
            return [left, right];
        } else if (sum < target) {
            left++; // necesitamos más
        } else {
            right--; // necesitamos menos
        }
    }
    return [];
}

// Two pointers para partition (荷兰国旗问题)
void partitionColors(List<int> arr) {
    int low = 0, mid = 0, high = arr.length - 1;

    while (mid <= high) {
        if (arr[mid] == 0) {
            swap(arr, low, mid);
            low++;
            mid++;
        } else if (arr[mid] == 1) {
            mid++;
        } else {
            swap(arr, mid, high);
            high--;
        }
    }
}
```

Error común: No ordenar el array primero cuando Two Pointers requiere orden.

3. BFS (Breadth-First Search)

Cuándo usar: Camino más corto en grafo sin peso, traversal por niveles, shortest path en grid.

Template:

```
// Tiempo:  $O(V + E)$  | Espacio:  $O(V)$ 
// V = vértices/nodos, E = aristas/conexiones
import 'dart:collection';

int bfs(List<List<int>> graph, int start, int target) {
  Queue<List<int>> queue = Queue(); // [node, distance]
  Set<int> visited = {};

  queue.add([start, 0]);
  visited.add(start);

  while (queue.isNotEmpty) {
    var current = queue.removeFirst();
    int node = current[0];
    int dist = current[1];

    if (node == target) return dist;

    for (int neighbor in graph[node]) {
      if (!visited.contains(neighbor)) {
        visited.add(neighbor);
        queue.add([neighbor, dist + 1]);
      }
    }
  }
  return -1; // no hay camino
}

// BFS en grid (Castle on the Grid style)
int bfsGrid(List<String> grid, int startX, int startY, int goalX, int goalY) {
  int n = grid.length;
  Queue<List<int>> queue = Queue();
  Set<(int, int)> visited = {};

  queue.add([startX, startY, 0]);
  visited.add((startX, startY));

  List<(int, int)> directions = [(0, 1), (0, -1), (1, 0), (-1, 0)];

  while (queue.isNotEmpty) {
    var current = queue.removeFirst();
    int x = current[0], y = current[1], steps = current[2];

    if (x == goalX && y == goalY) return steps;

    for (var (dx, dy) in directions) {
      int nx = x + dx, ny = y + dy;
      while (nx >= 0 && nx < n && ny >= 0 && ny < n && grid[nx][ny] == '.') {
        if (!visited.contains((nx, ny))) {
          visited.add((nx, ny));
          queue.add([nx, ny, steps + 1]);
        }
        nx += dx;
        ny += dy;
      }
    }
  }
  return -1;
}
```

Error común: Olvidar marcar visitados antes de agregar a la cola (causa duplicados).

4. DFS (Depth-First Search)

Cuándo usar: Explorar todos los caminos, contar componentes, detección de ciclos.

Template:

```
// Tiempo: O(V + E) | Espacio: O(V)
// DFS recursivo en graph
void dfs(List<List<int>> graph, int node, Set<int> visited) {
    visited.add(node);
    for (int neighbor in graph[node]) {
        if (!visited.contains(neighbor)) {
            dfs(graph, neighbor, visited);
        }
    }
}

// DFS en grid (contar islas)
int countIslands(List<List<int>> grid) {
    int count = 0;
    int rows = grid.length, cols = grid[0].length;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (grid[i][j] == 1) {
                dfsGrid(grid, i, j, rows, cols);
                count++;
            }
        }
    }
    return count;
}

void dfsGrid(List<List<int>> grid, int r, int c, int rows, int cols) {
    if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] == 0) return;

    grid[r][c] = 0; // marcar visitado
    dfsGrid(grid, r + 1, c, rows, cols);
    dfsGrid(grid, r - 1, c, rows, cols);
    dfsGrid(grid, r, c + 1, rows, cols);
    dfsGrid(grid, r, c - 1, rows, cols);
}
```

Error común: No backtrack (deshacer el cambio de estado) en problemas que lo requieren.

5. Binary Search

Cuándo usar: Array ordenado, buscar en espacio de respuesta monotónico.

Template:

```
// Tiempo: O(log n) | Espacio: O(1)
// Binary Search clásico
int binarySearch(List<int> arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) ~/ 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}

// Binary Search sobre respuesta
// "¿Es posible hacer X con calidad Y?"
bool isPossible(List<int> arr, int quality) {
    // retorna true si es factible con la calidad dada
    // ...
}

int searchAnswer(List<int> arr) {
    int left = 0, right = maxPossibleValue;

    while (left < right) {
        int mid = left + (right - left) ~/ 2;

        if (isPossible(arr, mid)) {
            right = mid; // buscar mejor
        } else {
            left = mid + 1;
        }
    }
    return left;
}
```

Error común: Off-by-one errors. Clarifica si `left < right` o `left <= right`.

6. Greedy

Cuándo usar: Elección local óptima lleva a óptimo global. Sin subproblemas superpuestos.

Template:

```
// Tiempo: O(n log n) por sorting | Espacio: O(1)
// Greedy con sorting
int greedySchedule(List<(int start, int end)> intervals) {
    intervals.sortBy((a, b) => a.$2.compareTo(b.$2)); // sort by end time

    int count = 0;
    int lastEnd = -1;

    for (var interval in intervals) {
        if (interval.$1 >= lastEnd) {
            count++;
            lastEnd = interval.$2;
        }
    }
    return count;
}

// Greedy con acumulador (Truck Tour style)
int findStart(List<(int petrol, int distance)> pumps) {
    int tank = 0;
    int start = 0;

    for (int i = 0; i < pumps.length; i++) {
        tank += pumps[i].$1 - pumps[i].$2;

        if (tank < 0) {
            start = i + 1;
            tank = 0;
        }
    }
    return start;
}
```

Error común: Asumir que greedy siempre funciona. Greedy requiere **probar** que la elección local es óptima (usualmente por contradicción o inducción).

7. Dynamic Programming (DP)

Cuándo usar: Subproblemas superpuestos + estructura de optimalidad.

Template:

```

// Tiempo:  $O(n)$  a  $O(n*m)$  según subproblemas | Espacio:  $O(n)$  a  $O(n*m)$ 
// DP bottom-up (tabulation)
int climbStairs(int n) {
    if (n <= 2) return n;

    List<int> dp = List.filled(n + 1, 0);
    dp[1] = 1;
    dp[2] = 2;

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}

// DP con memoization (top-down)
Map<String, int> memo = {};

int dpTopDown(int n) {
    if (n <= 2) return n;
    String key = '$n';
    if (memo.containsKey(key)) return memo[key]!;

    memo[key] = dpTopDown(n - 1) + dpTopDown(n - 2);
    return memo[key]!;
}

// DP 2D (Knapsack)
int knapsack(List<int> weights, List<int> values, int capacity) {
    int n = weights.length;
    List<List<int>> dp = List.generate(
        n + 1,
        (i) => List.filled(capacity + 1, 0),
    );

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            dp[i][w] = dp[i - 1][w]; // no tomar
            if (weights[i - 1] <= w) {
                dp[i][w] = max(
                    dp[i][w],
                    dp[i - 1][w - weights[i - 1]] + values[i - 1],
                );
            }
        }
    }
    return dp[n][capacity];
}

```

Error común: No definir bien la recurrence relation. Pregúntate: "¿Cuál es la subpregunta más pequeña?"

8. Backtracking

Cuándo usar: Generar todas las combinaciones/permutaciones bajo restricciones. n pequeño (≤ 20).

Template:

```

// Tiempo:  $O(2^n)$  o  $O(n!)$  | Espacio:  $O(n)$ 
// Backtracking base
void backtrack(List<int> candidates, List<int> current, int start) {
    // procesar current (es solución válida)
    print(current);

    for (int i = start; i < candidates.length; i++) {
        // pruning: skip si no puede llevar a solución
        if (i > start && candidates[i] == candidates[i - 1]) continue;

        current.add(candidates[i]);
        backtrack(candidates, current, i + 1);
        current.removeLast(); // backtrack
    }
}

// Permutaciones
void permute(List<int> nums) {
    List<bool> used = List.filled(nums.length, false);
    List<int> current = [];

    void backtrack() {
        if (current.length == nums.length) {
            print(List.from(current));
            return;
        }

        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;

            used[i] = true;
            current.add(nums[i]);
            backtrack();
            current.removeLast();
            used[i] = false;
        }
    }

    backtrack();
}

```

Error común: No hacer backtrack (olvidar `removeLast` o `used[i] = false`).

Para el árbol de decisión completo de patrones, ver [03-reconocimiento-patrones.md](#).

06 — Comunicación y Pseudocódigo

Resolver el problema es la mitad. Comunicar tu solución claramente es la otra mitad — especialmente en entrevistas técnicas.

Por qué importa la comunicación

En una entrevista técnica, el interviewer no solo evalúa si tu código funciona. Evalúa:

1. **¿Entiendes el problema?** — ¿Preguntas de clarificación?
2. **¿Puedes comunicar tu approach?** — ¿Explicas antes de codificar?
3. **¿Manejas edge cases?** — ¿Piensas en casos extremos?
4. **¿Tu código es limpio?** — ¿Nombres claros, estructura lógica?

Estructura de Presentación (5 minutos)

1. REFORMULAR (30 seg)
"Entiendo que me pides [reformulación del problema]."
2. ANALIZAR CONSTRAINTS (30 seg)
"El input puede tener hasta $n=[valor]$, así que necesito [complejidad]."
3. Mencionar brute force (30 seg)
"El approach más obvio sería [brute force] con complejidad $O(?)$.
Pero eso es demasiado lento porque [razón]."
4. Presentar el approach óptimo (1 min)
"Puedo mejorar esto usando [patrón/estructura]. La idea es [explicación en 2 oraciones]."
5. Pseudocódigo (2 min)
[Escribir pseudocódigo en la pizarra o en voz alta]

Cómo Escribir Pseudocódigo Limpio

Reglas

1. **Un paso por línea.** No metas lógica compleja en una línea.

2. **Usa nombres descriptivos.** `queue`, `visited`, `windowSum` — no `q`, `v`, `s`.
3. **Indentación para bloques.** While, for, if deben estar indentados.
4. **Comentarios para la lógica clave.** No para obviedades.
5. **Separa datos de lógica.** Primero la inicialización, luego el loop.

Ejemplo malo

```
while q not empty:
    x y d = q.pop(); if x==gx and y==gy return d; for dx dy in dirs: nx = x+dx... if valid add q
```

Ejemplo bueno

```
BFS(grid, start, goal):
    1. Cola = [(startX, startY, 0)]
    2. Visited = {(startX, startY)}

    3. Mientras cola no esté vacía:
        a. Extraer (x, y, pasos) de la cola
        b. Si (x, y) == goal, retornar pasos
        c. Para cada dirección (arriba, abajo, izq, der):
            i. nx, ny = siguiente celda en esa dirección
            ii. Mientras nx,ny sea válida y no visitada:
                - Marcar visitada
                - Agregar a cola con pasos + 1

    4. Retornar -1 (no hay camino)
```

Comunicación Durante la Implementación

Mientras codificas, narra lo que haces:

```
// "Voy a crear un HashMap para trackear los elementos que ya vi"
Map<int, int> seen = {};

// "Itero sobre el array una sola vez — O(n)"
for (int i = 0; i < arr.length; i++) {

    // "Calculo el complemento — esto es lo que necesito para sumar al target"
    int complement = target - arr[i];

    // "Si el complemento ya está en el map, encontré la respuesta"
    if (seen.containsKey(complement)) {
        return [seen[complement]!, i];
    }

    // "Si no, guardo el elemento actual con su índice"
    seen[arr[i]] = i;
}
```


El Mantra de la Entrevista

Antes de empezar a codificar, di:

"Elegí [patrón] porque [señal del enunciado], la complejidad es [O(?)] tiempo, O(?) espacio, los edge cases son [casos], las alternativas que consideré fueron [por qué no otras]."

Ejemplo completo

"Elegí **BFS** porque el problema pide **camino mínimo en un grid sin pesos de movimiento**, la complejidad es **O(N²) tiempo, O(N²) espacio**, los edge cases son **grid 1×1, start == goal, todo bloqueado**, las alternativas que consideré fueron **DFS (no garantiza mínimo) y Dijkstra (sobredimensionado para sin pesos)**."

Después de Codificar: Verificación

Siempre termina con:

- 1. **Testear con el ejemplo del enunciado** — traza el código paso a paso
- 2. **Mencionar edge cases** — "Esto maneja el caso donde [edge case] porque [razón]"
- 3. **Decir la complejidad final** — "Esta solución es O([complejidad]) en tiempo y O([complejidad]) en espacio"

Errores Comunes de Comunicación

Error	Cómo evitarlo
Codificar en silencio	Narra cada paso mientras escribes
No preguntar antes de empezar	"¿Puedo asumir que [asunción]?"
Brute force sin mencionar optimización	Siempre menciona el brute force primero
No validar con el interviewer	"¿Tiene sentido este approach hasta ahora?"
Olvidar edge cases	Menciona al menos 2 edge cases antes de codificar
No decir la complejidad	Siempre termina con "La complejidad es O(?)"

07 — Ejercicios de Práctica

10 ejercicios progresivos (3 easy, 4 medium, 3 hard) organizados por patrón. Cada uno incluye: enunciado, pistas, y solución en Dart.

Easy (3 ejercicios)

Ejercicio 1: Two Sum

Patrón: HashMap | **Plataforma:** LeetCode #1

Enunciado: Dado un array de enteros `nums` y un entero `target`, retorna los índices de dos números que sumen `target`.

Pistas:

- ¿Qué necesitas buscar para cada elemento? El complemento.
- ¿Cuál es la complejidad de buscar en un HashMap? $O(1)$.
- ¿Puedes resolverlo en un solo pass?

Solución en Dart:

```
List<int> twoSum(List<int> nums, int target) {
  Map<int, int> seen = {};

  for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (seen.containsKey(complement)) {
      return [seen[complement]!, i];
    }
    seen[nums[i]] = i;
  }

  return [];
}

// Tiempo: O(n) | Espacio: O(n)
```

Ejercicio 2: Valid Anagram

Patrón: HashMap (frecuencias) | **Plataforma:** LeetCode #242

Enunciado: Dados dos strings `s` y `t`, retorna `true` si `t` es un anagrama de `s`.

Pistas:

- Si tienen diferente longitud, no pueden ser anagramas.
- Cuenta frecuencias de cada carácter en ambos strings.
- ¿Las frecuencias deben ser iguales?

Solución en Dart:

```
bool isAnagram(String s, String t) {  
  if (s.length != t.length) return false;  
  
  Map<String, int> freq = {};  
  
  for (var ch in s.split('')) {  
    freq[ch] = (freq[ch] ?? 0) + 1;  
  }  
  
  for (var ch in t.split('')) {  
    freq[ch] = (freq[ch] ?? 0) - 1;  
    if (freq[ch]! < 0) return false;  
  }  
  
  return true;  
}  
// Tiempo: O(n) | Espacio: O(1) — máximo 26 caracteres
```

Ejercicio 3: Maximum Subarray (Kadane's Algorithm)

Patrón: DP lineal | **Plataforma:** LeetCode #53 | **Plantilla:** Ver [DP en 05-patrones-avanzados.md](#)

Enunciado: Dado un array de enteros, encuentra el subarray contiguo con la mayor suma.

Pistas:

- Para cada posición, ¿el subarray máximo que termina aquí incluye o no el elemento anterior?
- Si el acumulado anterior es negativo, empieza de nuevo.
- Mantén un `currentMax` y un `globalMax`.

Solución en Dart:

```
int maxSubArray(List<int> nums) {  
  int currentMax = nums[0];  
  int globalMax = nums[0];  
  
  for (int i = 1; i < nums.length; i++) {  
    currentMax = max(nums[i], currentMax + nums[i]);  
    globalMax = max(globalMax, currentMax);  
  }  
  
  return globalMax;  
}  
// Tiempo: O(n) | Espacio: O(1)
```

Medium (4 ejercicios)

Ejercicio 4: Best Time to Buy and Sell Stock

Patrón: One-pass, tracking mínimo | **Plataforma:** LeetCode #121

Enunciado: Dado un array `prices` donde `prices[i]` es el precio de una acción en el día `i`, encuentra la máxima ganancia que puedes lograr comprando y vendiendo una vez.

Pistas:

- Mantén el precio mínimo visto hasta ahora.
- Para cada día, calcula la ganancia si vendieras hoy.
- Actualiza la ganancia máxima.

Solución en Dart:

```
int maxProfit(List<int> prices) {
    int minPrice = prices[0];
    int maxProfit = 0;

    for (int i = 1; i < prices.length; i++) {
        if (prices[i] < minPrice) {
            minPrice = prices[i];
        } else {
            maxProfit = max(maxProfit, prices[i] - minPrice);
        }
    }

    return maxProfit;
}
// Tiempo: O(n) | Espacio: O(1)
```

Ejercicio 5: Number of Islands

Patrón: DFS/BFS en grid | **Plataforma:** LeetCode #200 | **Plantilla:** Ver [DFS en 05-patrones-avanzados.md](#)

Enunciado: Dado un grid de `'1'` (tierra) y `'0'` (agua), cuenta el número de islas. Una isla es formada por `'1'`s conectados horizontal o verticalmente.

Pistas:

- Recorre el grid. Cada `'1'` no visitado inicia una nueva isla.
- Usa DFS o BFS para "hundir" toda la isla (marcar como visitada).
- Incrementa el contador cada vez que encuentras un `'1'` nuevo.

Solución en Dart:

```

int numIslands(List<List<String>> grid) {
    int count = 0;
    int rows = grid.length, cols = grid[0].length;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '1') {
                dfs(grid, r, c, rows, cols);
                count++;
            }
        }
    }
    return count;
}

void dfs(List<List<String>> grid, int r, int c, int rows, int cols) {
    if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] != '1') return;

    grid[r][c] = '0'; // marcar visitado
    dfs(grid, r + 1, c, rows, cols);
    dfs(grid, r - 1, c, rows, cols);
    dfs(grid, r, c + 1, rows, cols);
    dfs(grid, r, c - 1, rows, cols);
}
// Tiempo: O(rows × cols) | Espacio: O(rows × cols) peor caso recursion

```

Ejercicio 6: Coin Change

Patrón: Dynamic Programming | **Plataforma:** LeetCode #322 | **Plantilla:** Ver [DP en 05-patrones-avanzados.md](#)

Enunciado: Dado un array de denominaciones de monedas `coins` y un `amount`, retorna el número mínimo de monedas para llegar a ese monto. Si no es posible, retorna -1.

Pistas:

- `dp[i]` = mínimo monedas para hacer el monto `i`.
- Para cada monto, prueba todas las monedas: `dp[i] = min(dp[i], dp[i - coin] + 1)`.
- Base: `dp[0] = 0` (0 monedas para monto 0).

Solución en Dart:

```
int coinChange(List<int> coins, int amount) {
    List<int> dp = List.filled(amount + 1, amount + 1);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin in coins) {
            if (coin <= i && dp[i - coin] + 1 < dp[i]) {
                dp[i] = dp[i - coin] + 1;
            }
        }
    }

    return dp[amount] > amount ? -1 : dp[amount];
}
// Tiempo: O(amount × coins.length) | Espacio: O(amount)
```

Ejercicio 7: LRU Cache

Patrón: HashMap + Doubly Linked List | **Plataforma:** LeetCode #146

Enunciado: Implementa una estructura de datos LRU (Least Recently Used) Cache con `get(key)` y `put(key, value)` en O(1).

Pistas:

- Usa un HashMap para acceso O(1) por key.
- Usa una doubly linked list para mantener el orden de uso.
- El más reciente va al final; el más viejo va al frente.
- Al hacer `get`, mueve el nodo al final. Al `put` con capacidad llena, elimina el frente.

Solución en Dart (esqueleto con LinkedHashMap):

```
class LRUCache {
    final int capacity;
    final LinkedHashMap<int, int> _cache;

    LRUCache(this.capacity)
        : _cache = LinkedHashMap();

    int get(int key) {
        if (!_cache.containsKey(key)) return -1;
        _refresh(key);
        return _cache[key]!;
    }

    void put(int key, int value) {
        if (_cache.containsKey(key)) {
            _cache.remove(key);
        } else if (_cache.length >= capacity) {
            _cache.remove(_cache.keys.first);
        }
        _cache[key] = value;
    }

    void _refresh(int key) {
        int value = _cache.remove(key)!;
        _cache[key] = value;
    }
}

// Tiempo: O(1) get y put | Espacio: O(capacity)
```

Hard (3 ejercicios)

Ejercicio 8: Merge K Sorted Lists

Patrón: Heap (PriorityQueue) | **Plataforma:** LeetCode #23

Enunciado: Dadas `k` linked lists ordenadas, merge todas en una sola linked list ordenada.

Pistas:

- Usa un min-heap con un elemento de cada lista.
- Extrae el mínimo, agrega al resultado, y agrega el siguiente de esa lista al heap.
- Complejidad: $O(N \log k)$ donde N es el total de elementos.

Solución en Dart (esqueleto):

```
ListNode? mergeKLists(List<ListNode?> lists) {
    var heap = PriorityQueue<ListNode>{(a, b) => a.val.compareTo(b.val)};

    for (var node in lists) {
        if (node != null) heap.add(node);
    }

    ListNode dummy = ListNode(0);
    ListNode current = dummy;

    while (heap.isNotEmpty) {
        ListNode node = heap.removeFirst();
        current.next = node;
        current = node;
        if (node.next != null) heap.add(node.next!);
    }

    return dummy.next;
}
// Tiempo: O(N log k) | Espacio: O(k)
```

Ejercicio 9: Alien Dictionary

Patrón: Topological Sort | **Plataforma:** LeetCode #269 | **Plantilla:** Ver [BFS en 05-patrones-avanzados.md](#)

Enunciado: Dado un array de palabras de un diccionario alienígena ordenado, determina el orden de los caracteres.

Pistas:

- Compara palabras adyacentes para encontrar precedencias (carácter A viene antes que B).
- Construye un grafo dirigido donde $A \rightarrow B$ significa "A va antes que B".
- Aplica topological sort (Kahn's algorithm con BFS).

Solución en Dart (esqueleto):


```

String alienOrder(List<String> words) {
    Map<String, Set<String>> adj = {};
    Map<String, int> inDegree = {};

    // Inicializar todos los caracteres
    for (var word in words) {
        for (var ch in word.split('')) {
            adj.putIfAbsent(ch, () => {});
            inDegree.putIfAbsent(ch, () => 0);
        }
    }

    // Construir grafo
    for (int i = 0; i < words.length - 1; i++) {
        String w1 = words[i], w2 = words[i + 1];
        int minLen = min(w1.length, w2.length);

        for (int j = 0; j < minLen; j++) {
            if (w1[j] != w2[j]) {
                String from = w1[j], to = w2[j];
                if (!adj[from]!.contains(to)) {
                    adj[from]!.add(to);
                    inDegree[to] = inDegree[to]! + 1;
                }
                break;
            }
        }
    }

    // Kahn's BFS
    Queue<String> queue = Queue();
    for (var entry in inDegree.entries) {
        if (entry.value == 0) queue.add(entry.key);
    }

    String result = '';
    while (queue.isNotEmpty) {
        String ch = queue.removeFirst();
        result += ch;
        for (var next in adj[ch]!) {
            inDegree[next] = inDegree[next]! - 1;
            if (inDegree[next] == 0) queue.add(next);
        }
    }

    return result.length == inDegree.length ? result : '';
}
// Tiempo: O(C) donde C es la longitud total de caracteres | Espacio: O(1) – máximo 26 letras

```

Ejercicio 10: Word Ladder

Patrón: BFS | **Plataforma:** LeetCode #127 | **Plantilla:** Ver [BFS en 05-patrones-avanzados.md](#)

Enunciado: Dado `beginWord`, `endWord` y un diccionario `wordList`, encuentra la longitud más corta de `beginWord` a `endWord` cambiando una letra a la vez. Cada palabra intermedia debe estar en el diccionario.

Pistas:

- Cada palabra es un nodo; dos palabras están conectadas si difieren en una letra.

- BFS desde `beginWord` hasta `endWord`.
- Para eficiencia, pre-computa un "patrón" para cada palabra ($h * (\text{wordLen})$).

Solución en Dart (esqueleto):

```
int ladderLength(String beginWord, String endWord, List<String> wordList) {
    Set<String> wordSet = Set.from(wordList);
    if (!wordSet.contains(endWord)) return 0;

    Queue<(String, int)> queue = Queue(); // (word, steps)
    Set<String> visited = {};

    queue.add((beginWord, 1));
    visited.add(beginWord);

    while (queue.isNotEmpty) {
        var (word, steps) = queue.removeFirst();

        for (int i = 0; i < word.length; i++) {
            for (var ch in 'abcdefghijklmnopqrstuvwxyz'.split('')) {
                if (ch == word[i]) continue;

                String next = word.substring(0, i) + ch + word.substring(i + 1);

                if (next == endWord) return steps + 1;

                if (wordSet.contains(next) && !visited.contains(next)) {
                    visited.add(next);
                    queue.add((next, steps + 1));
                }
            }
        }
    }

    return 0;
}

// Tiempo:  $O(M^2 \times N)$  donde M = longitud de palabra, N = número de palabras
```

Progresión sugerida

Semana 1: Ejercicios 1-3 (Easy) → 15-20 min cada uno
Semana 2: Ejercicios 4-5 (Medium) → 30-45 min cada uno
Semana 3: Ejercicios 6-7 (Medium) → 45-60 min cada uno
Semana 4: Ejercicios 8-10 (Hard) → 60-90 min cada uno

Regla de los 20 minutos: Si llevas 20 minutos sin avance, mira una pista (no la solución completa). Intenta otros 20 minutos. Si aún no, revisa la solución y vuelve a intentarla mañana.

09: Recursión y Backtracking

Recursión es cuando una función se llama a sí misma. Backtracking es recursión con "deshacer". Juntos resuelven problemas que parecen imposibles.

Por qué importa

Muchos problemas de algoritmos **solo se resuelven** con recursión o backtracking:

- Permutaciones, combinaciones, subconjuntos
- Laberintos y juegos (N-Queens, Sudoku)
- Árboles (casi todo en árboles es recursión)

Recursión: Los 3 pasos

1. Caso base

¿Cuándo para? Sin esto, llamada infinita.

2. Caso recursivo

¿Cómo reduces el problema?

3. ¿Qué cambia en cada llamada?

Algo debe acercarse al caso base.

```
factorial(5)
= 5 * factorial(4)
= 5 * 4 * factorial(3)
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1 ← caso base
= 120
```

Template recursión

```
int resolver(tipo input) {
    // 1. Caso base
    if (input es casoBase) return resultadoBase;

    // 2. Caso recursivo (reducir input)
    return operacion(input, resolver(inputReducido));
}
```

Ejemplo: Fibonacci

```
// ✗ Sin recursión: iterativo
int fibonacci(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        int temp = a + b;
        a = b;
        b = temp;
    }
    return b;
}

// ✔ Recursivo (simple pero lento  $O(2^n)$ )
int fibRecursivo(int n) {
    if (n <= 1) return n;
    return fibRecursivo(n - 1) + fibRecursivo(n - 2);
}

// ✔ Recursivo + memo (rápido  $O(n)$ )
int fibMemo(int n, [Map<int, int> memo = const {}]) {
    if (n <= 1) return n;
    if (memo.containsKey(n)) return memo[n];
    final resultado = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    memo[n] = resultado;
    return resultado;
}
```

Backtracking: Recursión + "Deshacer"

Backtracking = probar algo, y si no funciona, **revertir** el cambio y probar otra cosa.

```
Explorar laberinto:
1. Mover a casilla → marcada como visitada
2. Si no hay salida → DESMARCAR (deshacer)
3. Probar otra dirección
```

Template Backtracking

```
void backtracking(estado, candidatos) {
    // 1. ¿Es solución?
    if (esSolucion(estado)) {
        agregarSolucion(estado);
        return;
    }

    // 2. Probar cada candidato
    for (candidato in candidatos) {
        if (!esValido(candidato, estado)) continue;

        hacerMovimiento(candidato, estado);    // 3. Avanzar
        backtracking(estado, candidatos);      // 4. Explorar
        deshacerMovimiento(candidato, estado); // 5. Retroceder ← CLAVE
    }
}
```

Ejemplo: Permutaciones

```
List<List<int>>> permute(List<int> nums) {
    final resultado = <List<int>>>[];

    void backtrack(List<int> actual, List<bool> usados) {
        // Caso base: todos usados
        if (actual.length == nums.length) {
            resultado.add(List.from(actual));
            return;
        }

        for (int i = 0; i < nums.length; i++) {
            if (usados[i]) continue;

            // Hacer
            usados[i] = true;
            actual.add(nums[i]);

            // Explorar
            backtrack(actual, usados);

            // Deshacer ← CRUCIAL
            actual.removeLast();
            usados[i] = false;
        }
    }

    backtrack([], List.filled(nums.length, false));
    return resultado;
}

// Uso:
// permute([1, 2, 3])
// → [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
```

Ejemplo: Subconjuntos (Power Set)

```
List<List<int>> subsets(List<int> nums) {
    final resultado = <List<int>>[];

    void backtrack(int inicio, List<int> actual) {
        resultado.add(List.from(actual));

        for (int i = inicio; i < nums.length; i++) {
            actual.add(nums[i]);      // Incluir
            backtrack(i + 1, actual); // Explorar
            actual.removeLast();      // Excluir (deshacer)
        }
    }

    backtrack(0, []);
    return resultado;
}
```

Memoización: Evitar recalcular

Problema de recursión: recalcula lo mismo muchas veces.

```
// ✗ Sin memo: O(2^n) – lento
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}

// ✔ Con memo: O(n) – rápido
int fibMemo(int n, [Map<int, int> memo = const {}]) {
    if (n <= 1) return n;
    if (memo.containsKey(n)) return memo[n]!;
    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    return memo[n]!;
}
```

Regla: Si ves recursión con 2+ llamadas recursivas, necesitas memo.

Cuándo usar cada uno

Situación	Usar
Dividir y conquistar (merge sort)	Recursión simple
Explorar todas las combinaciones	Backtracking

Situación	Usar
Problema con subproblemas repetidos	Recursión + memoización
Árboles binarios	Recursión (DFS)

Errores comunes

Error	Solución
Sin caso base	Agrega <code>if (caso base) return;</code>
No deshacer cambios	Siempre <code>deshacer()</code> después de recursión
Memo olvidado	Usar <code>Map</code> o <code>@cache</code> de Dart
栈 overflow	Verificar que caso base se alcanza

Mini-ejercicio

Problema: Genera todas las combinaciones de k números del 1 al n.

```
List<List<int>>> combine(int n, int k) {  
  // Tu código aquí  
}
```

► Ver solución

Siguiente: [10-system-design-basico.md](#) - System Design para Flutter

10: System Design Básico para Flutter

System Design es diseñar un sistema completo (no solo una función). En entrevistas Flutter, evalúan si puedes diseñar una app real de forma escalable.

Por qué importa

Como Flutter developer, te preguntan:

- "Diseña un feed de Instagram"
- "Diseña un chat como WhatsApp"
- "Diseña un sistema de notificaciones push"

No es solo código Dart. Es **arquitectura completa**.

Framework de 4 pasos

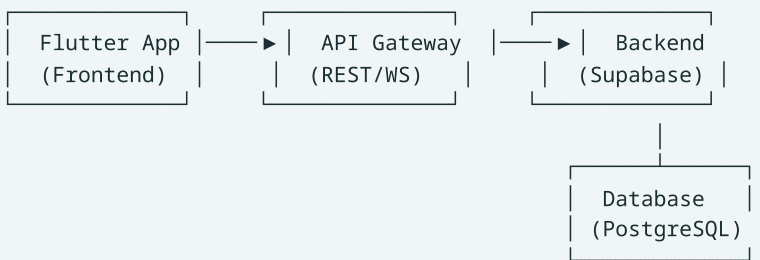
1. CLARIFICAR → Entender requisitos y restricciones
2. DISEÑAR → Alto nivel (componentes + datos)
3. PROFUNDIZAR → Detalles técnicos
4. ESCALAR → ¿Qué pasa con 1M usuarios?

Paso 1: Clarificar

Preguntas obligatorias:

- ¿Cuántos usuarios? (determina complejidad)
- ¿Cuántos datos por día? (determina almacenamiento)
- ¿Latencia aceptable? (determina caching)
- ¿Requiere real-time? (determina WebSocket vs polling)

Paso 2: Diseño de Alto Nivel



Componentes clave:

- **Ciente:** Flutter app (iOS, Android, Web)
- **API:** REST o WebSocket
- **Backend:** Supabase (auth, DB, storage, functions)
- **Base de datos:** PostgreSQL
- **Cache:** Redis o SharedPreferences local

Paso 3: Profundizar

Modelado de datos (ejemplo: Instagram feed)

```
-- Tabla de posts
CREATE TABLE posts (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  image_url TEXT NOT NULL,
  caption TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Tabla de likes (muchos a muchos)
CREATE TABLE likes (
  user_id UUID REFERENCES users(id),
  post_id UUID REFERENCES posts(id),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  PRIMARY KEY (user_id, post_id)
);

-- Índices para queries frecuentes
CREATE INDEX idx_posts_user ON posts(user_id);
CREATE INDEX idx_posts_created ON posts(created_at DESC);
CREATE INDEX idx_likes_post ON likes(post_id);
```

Flutter: Feed con pagination

```
class FeedCubit extends Cubit<FeedState> {
  final PostRepository _repo;
  static const int _pageSize = 20;

  FeedCubit(this._repo) : super(FeedInitial());

  Future<void> loadInitial() async {
    emit(FeedLoading());
    final posts = await _repo.getPosts(limit: _pageSize, offset: 0);
    emit(FeedLoaded(posts: posts, hasMore: posts.length == _pageSize));
  }

  Future<void> loadMore() async {
    if (state is! FeedLoaded) return;
    final current = state as FeedLoaded;
    final morePosts = await _repo.getPosts(
      limit: _pageSize,
      offset: current.posts.length,
    );
    emit(FeedLoaded(
      posts: [...current.posts, ...morePosts],
      hasMore: morePosts.length == _pageSize,
    ));
  }
}
```

Paso 4: Escalar

Problemas comunes y soluciones

Problema	Solución
Demasiadas consultas	Caching (Redis, local storage)
Imágenes lentas	CDN + compression
Real-time	WebSocket / Supabase Realtime
Base de datos lenta	Índices + read replicas
Un servidor saturado	Load balancing

Ejemplo: Notificaciones push

```
Supabase Database → Edge Function → Firebase Cloud Messaging → Flutter App
                    (trigger)           (envía push)
```

Template de respuesta en entrevista

```
## 1. Requisitos
- Usuarios: [X]
- Datos por día: [X]
- Latencia: [< X ms]

## 2. Diagrama
[dibujo de componentes]

## 3. Modelado
[tablas SQL + relaciones]

## 4. Flutter
[Cubit/BLoC + Repository]

## 5. Escalabilidad
[cacheing, CDNs, replicas]
```

Ejemplo resuelto: Diseña un Chat

1. Requisitos

- Mensajes en tiempo real
- Historial persistente
- 1:1 y grupal
- 10K usuarios concurrentes

2. Modelado

```
CREATE TABLE conversations (
  id UUID PRIMARY KEY,
  is_group BOOLEAN DEFAULT FALSE,
  name TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE messages (
  id UUID PRIMARY KEY,
  conversation_id UUID REFERENCES conversations(id),
  sender_id UUID REFERENCES users(id),
  content TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE conversation_members (
  conversation_id UUID REFERENCES conversations(id),
  user_id UUID REFERENCES users(id),
  PRIMARY KEY (conversation_id, user_id)
);
```

3. Flutter

```
class ChatCubit extends Cubit<ChatState> {
  final SupabaseClient _supabase;
  late RealtimeChannel _channel;

  ChatCubit(this._supabase) : super(ChatInitial());

  void subscribeToMessages(String conversationId) {
    _channel = _supabase
      .channel('messages:$conversationId')
      .onPostgresChanges(
        event: PostgresInsertEvent,
        schema: 'public',
        table: 'messages',
        filter: PostgresChangeFilter(
          type: PostgresChangeFilterType.eq,
          column: 'conversation_id',
          value: conversationId,
        ),
        callback: (payload) {
          final msg = Message.fromJson(payload.newRecord);
          emit(state.copyWith(messages: [...state.messages, msg]));
        },
      )
      .subscribe();
  }

  @override
  Future<void> close() {
    _supabase.removeChannel(_channel);
    return super.close();
  }
}
```

Errores comunes en entrevistas

Error	Solución
Empezar a codificar sin clarificar	Siempre preguntar primero
No mencionar escalabilidad	Agregar paso 4 siempre
Ignorar edge cases	Mencionar errores y límites
Solo REST cuando piden real-time	Mencionar WebSocket/Supabase Realtime

Siguiente: [11-errores-comunes-patron.md](#)

11: Errores Comunes al Resolver Problemas

Patrones de error que comete todo el mundo. Conocerlos te ayuda a detectarlos en tiempo real.

Errores por fase del proceso

Fase 1: Entender el problema

Error	Ejemplo	Solución
No leer todo el enunciado	Ignorar restricciones de $O(1)$ espacio	Leer 2 veces, subrayar restricciones
Asumir sin preguntar	"¿Puede haber duplicados?" → no pregunta	Siempre preguntar edge cases
Confundir 输入/输出	Retornar índices cuando piden valores	Releer: ¿qué debe retornar?

Fase 2: Elegir approach

Error	Ejemplo	Solución
Brute force sin pensar	Probar todas las permutaciones $O(n!)$	Buscar patrón primero
Over-engineering	Usar DP cuando con sorting basta	Empezar por la solución más simple
Ignorar restricciones	$O(n^2)$ cuando dice $O(n \log n)$	Ver límites antes de diseñar

Fase 3: Implementar

Error	Ejemplo	Solución
Off-by-one	<code>i <= n</code> vs <code>i < n</code>	Dibujar el caso base en papel
Índices inválidos	<code>array[array.length]</code>	Verificar bounds antes de acceder
Olvidar edge cases	Input vacío, null, un solo elemento	Testear: vacío, 1 elemento, 2 elementos
Mutar sin restaurar	No deshacer en backtracking	Siempre <code>deshacer()</code> después de recursión

Fase 4: Verificar

Error	Ejemplo	Solución
No probar con ejemplos	Solo mirar el código	Ejecutar mentalmente con el ejemplo dado
No verificar edge cases	Solo probar el caso feliz	Probar: vacío, 1 elemento, máximo
Asumir que funciona	"Se ve bien"	Dry run paso a paso

Errores por tipo de problema

Arrays/Strings

```
// × Error: No verificar si está vacío
int firstElement(List<int> arr) => arr[0]; // Crash si vacío

// ✔ Correcto
int? firstElement(List<int> arr) => arr.isEmpty ? null : arr[0];
```

Hash Maps

```
// × Error: No verificar si existe la key
int value = map[key!]; // Crash si no existe

// ✔ Correcto
int? value = map[key]; // Null si no existe
```

Recursión

```
// × Error: Caso base incorrecto
int factorial(int n) => n * factorial(n - 1); // Infinito

// ✔ Correcto
int factorial(int n) {
  if (n <= 1) return 1; // Caso base
  return n * factorial(n - 1);
}
```

Backtracking

```
// ✗ Error: No deshacer
void backtrack(List<int> actual, int i) {
    actual.add(i);          // Avanzar
    backtrack(actual, i + 1);
    // Falta: actual.removeLast()
}

// ✔ Correcto
void backtrack(List<int> actual, int i) {
    actual.add(i);          // Avanzar
    backtrack(actual, i + 1);
    actual.removeLast();    // ← DESHACER
}
```

Checklist de verificación

Antes de decir "estoy listo":

- ☐ ¿Leí TODAS las restricciones?
- ☐ ¿Probé con el ejemplo del enunciado?
- ☐ ¿Probé edge cases? (vacío, 1 elem, null)
- ☐ ¿La complejidad cumple el requisito?
- ☐ ¿El retorno es correcto? (índices vs valores)
- ☐ ¿El código compila en Dart?

Regla de los 3 casos

Siempre prueba tu solución con:

1. Caso vacío/null → ¿No crashea?
2. Caso mínimo (1 elem) → ¿Funciona?
3. Caso normal (ejemplo) → ¿Retorna correcto?

Siguiente: [12-ejercicios-adicionales.md](#) - +25 ejercicios para practicar

12: Ejercicios Adicionales (+25 problemas)

Ejercicios organizados por patrón y dificultad. Usa el framework de 6 pasos del 01-metodologia-general.md para cada uno.

Cómo usar este archivo

1. Elige un ejercicio por dificultad
2. Aplica los 6 pasos (lee → identifica → diseña → implementa → verifica → optimiza)
3. No mires la solución hasta haber intentado 15 minutos
4. Si te trabas 30 minutos, mira la pista, no la solución completa

Nivel 1: Fácil (1-2 pasos)

1. Two Sum

Dado un array y un target, encuentra dos números que sumen el target. Retorna sus índices.

```
Input: nums = [2, 7, 11, 15], target = 9  
Output: [0, 1]
```

Pista: ¿Qué necesitas para encontrar complemento?

► Solución

2. Valid Anagram

Determina si dos strings son anagramas.

```
Input: s = "anagram", t = "nagaram"  
Output: true
```

Pista: ¿Qué pasa con las frecuencias de caracteres?

► Solución

3. Reverse String

Invierte un array de caracteres in-place.

```
Input: ['h','e','l','l','o']  
Output: ['o','l','l','e','h']
```

► Solución

4. Merge Sorted Arrays

Fusiona dos arrays ordenados en orden ascendente.

```
Input: nums1 = [1,2,3], nums2 = [2,5,6]  
Output: [1,2,2,3,5,6]
```

► Solución

5. Contains Duplicate

Retorna true si algún elemento aparece dos veces.

```
Input: [1,2,3,1]  
Output: true
```

► Solución

6. Max Subarray Sum

Encuentra el subarray contiguo con mayor suma.

```
Input: [-2,1,-3,4,-1,2,1,-5,4]  
Output: 6 (subarray [4,-1,2,1])
```

Pista: Kadane's Algorithm

► Solución

Nivel 2: Medio (2-3 pasos)

7. Group Anagrams

Agrupar strings que son anagramas entre sí.

```
Input: ["eat", "tea", "tan", "ate", "nat", "bat"]
Output: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
```

► Solución

8. Longest Consecutive Sequence

Encuentra la secuencia consecutiva más larga.

```
Input: [100,4,200,1,3,2]
Output: 4 (secuencia [1,2,3,4])
```

Pista: Solo empieza a contar si `num - 1` no existe.

► Solución

9. Product of Array Except Self

Retorna array donde cada elemento es el producto de todos excepto sí mismo.

```
Input: [1,2,3,4]
Output: [24,12,8,6]
```

Pista: Producto izquierdo × Producto derecho.

► Solución

10. Valid Parentheses

Determina si los paréntesis son válidos.

```
Input: "([)]{}"
Output: true
```

► Solución

11. Binary Search Rotated

Busca un target en un array rotado ordenado.

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

► Solución

12. Climbing Stairs

¿De cuántas formas puedes subir n escalones? (1 o 2 pasos a la vez)

```
Input: n = 5
Output: 8
```

Pista: Es Fibonacci.

► Solución

Nivel 3: Difícil (3+ pasos)

13. Merge K Sorted Lists

Fusiona k listas ordenadas en una sola lista ordenada.

Pista: Usa un min-heap (pq de Dart: `PriorityQueue`).

► Solución (idea)

14. LRU Cache

Implementa una cache con capacidad fija que elimina el menos recientemente usado.

Pista: `LinkedHashMap` de Dart.

► Solución

15. Word Ladder

Encuentra el camino más corto transformando una palabra en otra cambiando una letra a la vez.

```
Input: begin = "hit", end = "cog", wordList = ["hot","dot","dog","lot","log","cog"]
Output: 5 (hit → hot → dot → dog → cog)
```

Pista: BFS (Breadth-First Search).

16. Median of Two Sorted Arrays

Encuentra la mediana de dos arrays ordenados en $O(\log(m+n))$.

Pista: Binary search en el array más corto.

17. Regular Expression Matching

Implementa `.` (cualquier char) y `*` (cero o más del anterior).

Pista: Recursión con memo.

Ejercicios Dart Específicos

18. Implementa un Stack usando List

```
class Stack<T> {  
  // Implementa push, pop, peek, isEmpty, size  
}
```

19. Implementa un Queue usando List

```
class Queue<T> {  
  // Implementa enqueue, dequeue, peek, isEmpty  
}
```

20. Implementa un HashMap simple

```
class SimpleHashMap<K, V> {  
  // Implementa put, get, remove, containsKey  
  // Usa un array de buckets (linked lists para colisiones)  
}
```

21. Binary Tree: DFS in-order, pre-order, post-order

```
class TreeNode<T> {  
  T value;  
  TreeNode<T>? left, right;  
  TreeNode(this.value);  
}  
  
// Implementa los 3 recorridos DFS
```

22. Binary Tree: BFS (nivel por nivel)

```
// Retorna listas de valores por nivel  
List<List<int>> levelOrder(TreeNode<int>? root) {  
  // Tu código aquí  
}
```

23. Graph: BFS desde un nodo

```
// Dado un grafo (adjacency list), retorna BFS
List<int> bfs(Map<int, List<int>> graph, int start) {
    // Tu código aquí
}
```

24. Graph: Detectar ciclo en directed graph

```
bool hasCycle(Map<int, List<int>> graph) {
    // Tu código aquí (DFS con estado: blanco, gris, negro)
}
```

25. Implementa Merge Sort

```
List<int> mergeSort(List<int> nums) {
    // Tu código aquí
}
```

26. Implementa Quick Sort

```
List<int> quickSort(List<int> nums) {
    // Tu código aquí
}
```

Progresión sugerida

```
Semana 1: Ejercicios 1-6 (fácil, 30 min cada uno)
Semana 2: Ejercicios 7-12 (medio, 45 min cada uno)
Semana 3: Ejercicios 13-17 (difícil, 60 min cada uno)
Semana 4: Ejercicios 18-26 (Dart específico, 30 min cada uno)
```

Total: ~26 ejercicios × 40 min promedio = ~17 horas

Volver al índice: [README.md](#)

08 — Recursos Externos

Libros, plataformas y repositorios para seguir practicando después de esta guía.

Libros Recomendados

Libro	Autor	Por qué
Competitive Programmer's Handbook	Antti Laaksonen	Gratuito. Cubre desde fundamentos hasta DP, grafos, matemáticas. cses.fi/book/book.pdf
Principles of Algorithmic Problem Solving	Martin M. M. Guzman	Metodología + ejemplos resueltos. USACO Guide
Guide to Competitive Programming	Kaarto y Laaksonen	Springer. Cubre IOI syllabus completo.
Programming Challenges	Skiena y Revilla	El clásico para empezar en competitive programming
Introduction to Algorithms (CLRS)	Cormen et al.	Referencia académica completa de algoritmos

Plataformas de Práctica

Por nivel de dificultad

Plataforma	Nivel ideal	Tipo de problemas
Exercism	Principiante	Fundamentos de lenguaje, Dart track disponible
Codewars	Principiante-Intermedio	Katas cortas, progresivas
HackerRank	Intermedio	Interview Prep Kit, data structures, algorithms
LeetCode	Intermedio-Avanzado	El estándar para entrevistas técnicas
Codeforces	Avanzado	Competencias en vivo, problemas clasificados por rating
CSES Problem Set	Intermedio-Avanzado	300+ problemas estructurados por tema

Rutas de aprendizaje recomendadas

Para entrevistas (3 meses):

Mes 1: HackerRank Interview Prep Kit (warm-up, arrays, strings)

Mes 2: LeetCode NeetCode 150 (patrón por patrón)

Mes 3: LeetCode Grind 75 (los 75 más frecuentes en FAANG)

Para competitive programming (6 meses):

Mes 1-2: CSES Problem Set (Introduction + Sorting + Graphs)

Mes 3-4: Codeforces Div 2 A-C problems

Mes 5-6: Codeforces Div 2 D-E + AtCoder ABC

Repositorios Útiles en GitHub

Repositorio	Contenido
neetcodeio/neetcode	NeetCode 150 con soluciones en múltiples lenguajes
kamyu104/LeetCode-Solutions	Soluciones en Python/C++ de todos los LeetCode
biswas-07/CSES-Problem-Set-Solutions	Soluciones del CSES Problem Set
simranjeet97/DSA_Complete_Pattern_Recognition_Guide	Guía completa de reconocimiento de patrones

Canales de YouTube

Canal	Tipo de contenido
NeetCode	Explicaciones de LeetCode NeetCode 150 con diagramas
take U forward (Striver)	LeetCode Grind 75, explicaciones paso a paso
Abdul Bari	Fundamentos de algoritmos con visualizaciones
William Fiset	Graph algorithms, DSU, advanced topics
Errichto	Competitive programming, live solving

Consejos para la práctica

1. **Consistencia > cantidad.** 1 problema diario > 20 problemas un sábado.

2. **Repite problemas.** Si resolviste algo hace 30 días, resuélvelo de nuevo. Si no puedes, no lo aprendiste.
3. **Explica en voz alta.** Después de resolver, narrá la solución como si le explicaras a alguien. Si te trabás, no lo dominás.
4. **Respeta el timebox.** 20 minutos stuck = mirar pista. 40 minutos stuck = mirar solución. Volver a intentar al día siguiente.
5. **Agrupar por patrón, no por dificultad.** Resolver 5 problemas de Sliding Window seguidos enseña más que 5 problemas random.
6. **Trackea tu progreso.** Usa una spreadsheet: problema, patrón, si lo resolviste solo, días desde la última revisión.